

Institut für Parallele und Verteilte Systeme

Universität Stuttgart
Universitätsstraße 38
D-70569 Stuttgart

Masterarbeit

A Co-Design Framework for Networked Control Systems in Time-Sensitive Networks

Srinidhi Krishna, Kalpathy Sundaram

Studiengang: Computer Science

Prüfer/in: Prof. Dr. Christian Becker

Betreuer/in: Simon Egger, M.Sc.

Beginn am: November 20, 2025

Beendet am: May 20, 2026

Abstract

Network Control Systems (NCS) require the timely closure of the feedback loop while operating across a shared network that introduces delay and packet loss. System safety must be robust to these network imperfections, and substantial work has been done to derive stability bounds for NCS under fixed operating conditions. However, many practical systems require dynamic adaptation to operating conditions, either due to external disturbances or intentional switching, which necessitates simultaneous changes in control and network parameters. In practice, this switch can be triggered prematurely, resulting in a transition window during which system safety is not guaranteed. Previous work on switched systems focuses on control signal switching without considering network reconfiguration, and existing NCS stability analysis do not model the transition window or derive conditions for its safety. We formulate a sufficient condition that bounds the worst-case open-loop duration a plant can safely sustain across a mode transition using the network delivery guarantee, the reconfiguration delay, and the control stability tolerance. We develop a Timed Colored Petri Net (TCPN) validation framework that simulates packet flow, mode switching, and reconfiguration delay to report whether the system remains safe across different operating conditions. The framework is evaluated on an inverted pendulum control system, confirming the soundness of the sufficient condition and demonstrating the framework as a co-design tool to explore the trade-off between reconfiguration delay, network guarantee, and control-side stability tolerance.

Contents

Abbreviations	13
1 Introduction	15
2 Background	17
2.1 Network Control System	17
2.2 Weakly Hard Real-Time Guarantees	20
2.3 Petri Nets	21
3 Related Works	25
3.1 Stability of NCS	25
3.2 Validation Frameworks for NCS	26
4 System Model and Problem Statement	29
4.1 System Model	29
4.2 Problem Statement	33
5 Framework Design	35
5.1 Mode Switching Model	35
5.2 Formal Model: System Model to TCPN	37
5.3 Mode Switch Simulation	43
5.4 Reporting	46
5.5 Validation	47
5.6 Implementation	48
5.7 Example: Configuration to TCPN model	49
6 Evaluation	53
6.1 Experimental Setup	53
6.2 Single Mode Stability	57
6.3 Mode Switch Evaluation	59
6.4 Switch Time Analysis	61
7 Conclusion	65

Bibliography	67
Appendix	72
A Use of AI Tools	73

List of Figures

1.1	General architecture for Network Control Systems	15
2.1	Effect of network delay on control packet delivery. A hit (a) updates u within the period; a miss (b) forces the actuator to hold the stale control until the next period.	19
2.2	Petri Net model, before and after firing	21
2.3	TCPN model, before and after firing	23
5.1	Transition window considering the worst-case of <i>miss</i> from network	36
5.2	TCPN framework pipeline	37
5.3	TCPN transitions for periodic stream generation and network scheduling.	40
5.4	TCPN transitions for end system forwarding and packet delivery.	42
5.5	Mode π_i : stream 1 active, stream 2 inactive.	45
5.6	Mode π_j : stream 2 active, stream 1 inactive.	46
5.7	CPN constructed from the minimal configuration in Listing 5.1.	50
6.1	Inverted pendulum control system	54
6.2	Maximum Consecutive Misses vs Sampling Period	56
6.3	Stability analysis for π_1	58
6.4	Stability analysis for π_2	59
6.5	Best-Case: mode switch when network guarantees $\mathcal{S}_{\text{net}, i} = 0$	60
6.6	Mode switch analysis under different network schedule	60
6.7	Effect of switch time on safety of system during mode switch from $\pi_1 \rightarrow \pi_2$	61
6.8	Effect of switch on safety of system during mode switch from $\pi_2 \rightarrow \pi_1$	62

List of Tables

5.1	Stream places in $\mathcal{P}_{\text{stream}}$ and their associated color sets.	40
5.2	Mapping of JSON fields to System Model	50
5.3	Marking evolution for the first three packets under schedule $\text{sched}(1) = (1, 0)$. .	51
6.1	LQR weight matrices per control mode.	55
6.2	LQR tuning parameters per control mode.	55

Listings

5.1	Minimal system configuration	49
6.1	Configuration for the two mode pendulum cart system	57

Abbreviations

Notation	Description	Page List
AGV	Automated Guided Vehicle	33
ES	End System	29, 33, 43, 50, 56, 57
LQR	Linear Quadratic Regulator	18, 30, 55, 57
LTI	Linear Time-Invariant	16, 17, 19, 25, 26, 32, 33, 53, 65
NCS	Network Control System	15–17, 19, 25–27, 29, 30, 37, 65
NN	Network Node	29, 56
PID	Proportional Integral Derivative	18

Abbreviations

Notation	Description	Page List
TCPN	Timed-Colored Petri Nets	5, 7, 16, 17, 22, 23, 26, 33, 35, 37, 39–41, 44, 48–52, 65, 66
TSN	Time-Sensitive Networking	25, 26
WHRT	Weakly Hard Real Time	16, 17, 20, 21, 25, 26, 30
ZOH	Zero-Order-Hold	18, 30, 36, 57

1 Introduction

A Network Control System (NCS) consists of spatially distributed sensors, controllers, and actuators connected through a shared communication network, as shown in Figure 1.1 [23]. NCS has applications in a broad range of areas, including space exploration, industrial automation, aircraft, and automobiles [23, 31, 43]. These systems require timely feedback to maintain their dynamics [6]. Hence, it is essential for the system to maintain bounded latency and low jitter on the feedback signals. In NCS, the controller computes an input to the plant based on the sensor measurements and transmits it to the actuator through the network. It is essential for system stability that the control packets arrive in a timely manner. However, the communication network that closes the loop is not ideal, and packets are bound to experience delays and loss as they traverse through the network [42]. The stability of the control system under network imperfections has been studied in [23, 37, 42, 49].

In practice, many NCS require the ability to adapt according to changes. For example, aircraft avionics systems must adapt their feedback loop during turbulence and normal operation. The feedback loop would have different stability requirements during turbulence and normal operation. Hence, these systems should be able to adapt their operating environment dynamically at runtime. The different operations can be formalized as multiple modes of operation. Each mode of operation corresponds to a system behavior and stability requirement [11]. A mode change

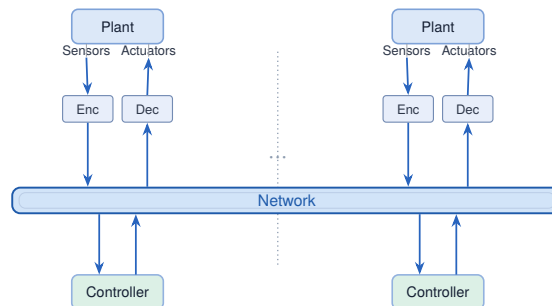


Figure 1.1: General architecture for Network Control Systems

involves reconfiguration on both the plant and network sides. The plant uses a sensor sampling period and control parameters, and the network needs to reconfigure its schedule to meet the stability requirements of the new mode.

The transition of modes of operation can be executed with synchronization, the system waits until the plant and network have reconfigured to switch to the next mode. However, mode transitions can be triggered prematurely due to plant disturbances or network fallback to control traffic. In such scenarios, one side has already switched while the other continues to operate in the old configuration, resulting in a window without stability guarantee. Hence, it is essential to bound the reconfiguration delay and determine the duration for which the plant remains stable during the transition.

To address this gap, we formulate a sufficient condition that maps the network guarantee and reconfiguration delay to the control-side stability requirement. We then develop a Timed-Colored Petri Nets (TCPN) [26] based validation framework that simulates the flow of packets as timed-colored tokens traversing through the network and reports whether the delivery trace of the packets satisfies the sufficient condition. The framework models mode switching and reconfiguration, enabling us to analyze the trade-off between reconfiguration delay and control stability.

This thesis makes the following contributions:

1. A sufficient condition for the stability of Linear Time-Invariant (LTI) system during mode transitions, expressed using network delivery guarantee, schedule reconfiguration delay, and control side tolerance.
2. A TCPN validation framework that simulates the mode transitions and observes the transition window and reports the violation of the sufficient condition.
3. An experimental evaluation of the framework on an inverted pendulum system to demonstrate the mode transition and reconfiguration delay, and analyze the boundary of the stability and the conservatism of the sufficient condition.

The remainder of this work is structured as follows. Chapter 2 provides background on NCS, stability under communication delay, Weakly Hard Real Time (WHRT) guarantees, and Petri Nets. Chapter 3 explores existing research on NCS stability and validation frameworks, and bridges the research gap we aim to explore. Chapter 4 defines the system model, modes of operation, and the problem statement. Chapter 5 formalizes mode switch and defines a sufficient condition for stability. We then present the formal design of TCPN framework. Chapter 6 evaluates the framework on the inverted pendulum on a cart control system. Finally, Chapter 7 concludes and discusses future work.

2 Background

This chapter provides an overview of the topics introduced in this work. In Section 2.1, we introduce the dynamics of the NCS under LTI, the discretization and stability of the system under communication delay. Section 2.2 defines the WHRT guarantee model based on the existing literature. Section 2.3 introduces Petri Nets and defines the formal definition of TCPN.

2.1 Network Control System

NCS definitions across literature define it as an exchange of information, i.e, plant output, control input, and control system components through a communication network [17]. The sensors sense plant's state and transmit it to the controller over the network. The control computes an appropriate response to be applied to the plant and transmits the control data to the actuator through the same network. Actuators receive the control information and control the plant accordingly. In such closed-loop systems, the controller aims to reduce the error and bring the plant to the desired state [39].

2.1.1 Plant

A plant may be any piece of equipment whose purpose is to perform a set of operations. The plant dynamics can be described as a continuous LTI as described in [18] as

$$\frac{d}{dt}x(t) = Ax(t) + Bu(t) \quad (2.1)$$

$$y(t) = Cx(t) + Du(t) \quad (2.2)$$

where $x(t) \in \mathbb{R}^n$ is the plant state vector at time t , $u(t) \in \mathbb{R}^m$ is the control input and $y(t)$ is the measured control output. The matrices A, B, C, D represent the dynamics of the system. Throughout this work, we assume D is a zero matrix, and C is a unit matrix of appropriate size¹.

¹ A is $n \times n$ matrix, B is $n \times m$ matrix, C is $n \times n$ matrix, D is $n \times m$ matrix

2 Background

In practice, the current state of a system is usually transmitted to the controller at a fixed sampling rate. This motivates us to treat the system as discrete. The system samples the current state at discrete times and the corresponding control signal is applied at discrete times [6].

2.1.2 Discretization

In the discrete system, we assume that the system state is sampled periodically with an interval $h > 0$, resulting in a sequence of sampling instants $t_k = k \cdot h$ for $k \in \mathbb{Z}^+$. At each sampling instant t_k , the sensor measures the plant state $x[k]$ [6] [23]. Given the sampling time t_k , the state at a future time $t \in [t_k, t_{k+1})$ is described as

$$\begin{aligned} x(t) &= e^{A(t-t_k)}x(t_k) + \int_{t_k}^t e^{A(t-\tau)}B u(\tau) d\tau \\ &= e^{A(t-t_k)}x(t_k) + \int_{t_k}^t e^{A(t-\tau)} d\tau B u(t_k) \end{aligned} \quad (2.3)$$

Equation (2.3) is derived as the control is constant between $[t_k, t_{k+1})$. This model of constant control between the sampling instants is called Zero-Order-Hold (ZOH) [6]. Since the control input is only updated at the sampling instants, the interval between two consecutive sampling points is when the system is in open loop.

The state at $(k+1)^{th}$ instant can be represented as a difference equation, i.e.,

$$x[k+1] = A_d x[k] + B_d u[k] \quad (2.4)$$

where $A_d = e^{Ah}$ and $B_d = \int_0^h e^{A\tau} d\tau B$.

The control input $u[k]$ that stabilizes the plant can be computed through different methods. The classical Proportional Integral Derivative (PID) computes u as the weighted sum of the proportional, integral, and derivative terms. PIDs are widely used in industries, as most systems can be controlled, provided the performance requirement is low. PID controllers are generally well suited for lower-order systems. When the system is of higher order, the control performance can be improved with a complex controller [5]. Additionally, methods like pole-placement can be used to solve the control design problems [33].

The PID belongs to a class of sub-optimal controllers. Alternatively, the controller can be designed to have optimal control. The Linear Quadratic Regulator (LQR) is an optimal control method that calculates the gain matrix K by minimizing the quadratic cost function [32].

$$J = \sum_{k=0}^{\infty} (x[k]^T Q x[k] + u[k]^T R u[k]) \quad (2.5)$$

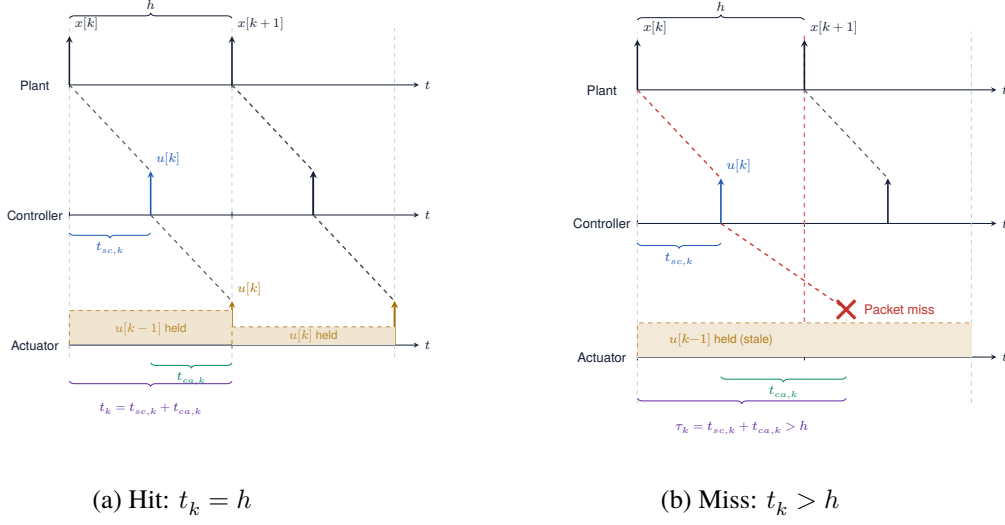


Figure 2.1: Effect of network delay on control packet delivery. A hit (a) updates u within the period; a miss (b) forces the actuator to hold the stale control until the next period.

for given positive definite weight matrices Q and R . The optimal gain is computed by solving the variable P in the Algebraic Ricatti Equation [29].

2.1.3 Stability Analysis Under Communication Delay

The fundamental assumption in control theory is that the communication channels are ideal. In NCS, the control loop is closed by unreliable communication channels. The data sampled from the sensor, and control information to the actuator are prone to network disturbances. These perturbations affect the stability of the system. Protocols like TCP provide an eventual guarantee, but in NCS, outdated packets are often not useful [23].

The stability of the control system is affected when it is designed without considering these delays [49]. Figure 2.1 shows two sources of delays in NCS. The delay induced by the network between the sensor and controller t_{sc} , and the delay between controller and actuator t_{ca} . The delay from the computation of the controller can be modeled into t_{ca} or t_{sc} , then the net delay from the network is $t = t_{sc} + t_{ca}$ [38]. For the k^{th} sampling instant in Figure 2.1a, the network delay $t \leq h$ (*hit*), and the control force applied on the plant at the $k + 1$ instant is $u[k]$ representing the discrete LTI dynamics from Equation (2.4). In Figure 2.1b, the delay $t > h$ (*miss*), and the control force applied for the $k + 1$ instant is the stale control from $u[k - 1]$. Hence, understanding the plant dynamics under a *hit*, and a *miss* is essential to characterise the extent to which the system remains stable when packets *miss* the deadline.

2 Background

This system needs to be treated as an augmented state vector, which inputs past values of the plant state and output of the current plant values [18]. In [37], the augmented state equation under the *hit* and *miss* has been studied, and the upper bound on the control update miss has been derived. The plant dynamics when the deadlines of the packets are not missed, and the control loop is closed are

$$\begin{bmatrix} x[k] \\ u[k] \end{bmatrix} = \underbrace{\begin{bmatrix} A_d & B_d \\ -K & 0_{r,r} \end{bmatrix}}_{A_{\text{hit}}} \begin{bmatrix} x[k-1] \\ u[k-1] \end{bmatrix} \quad (2.6)$$

The system equation when the deadline is missed is described by

$$\begin{bmatrix} x[k] \\ u[k] \end{bmatrix} = \underbrace{\begin{bmatrix} A_d & B_d \\ 0 & I_{r,r} \end{bmatrix}}_{A_{\text{miss}}} \begin{bmatrix} x[k-1] \\ u[k-1] \end{bmatrix} \quad (2.7)$$

The evolution of the system under i consecutive control update misses is derived as

$$Z_{k+i+1} = A_{\text{miss}}^i \cdot A_{\text{hit}} \cdot Z_k \quad (2.8)$$

In [47], the Schur stability criterion [6] is extended to represent the sufficient condition for stability under consecutive packet misses as

$$\max_l |\lambda_l (A_{\text{miss}}^i A_{\text{hit}})| < 1 \quad (2.9)$$

2.2 Weakly Hard Real-Time Guarantees

A WHRT system is a system that can tolerate occasional losses of deadlines. The distribution of *hits* and *misses* on deadline over a timed window is bounded [8]. The (m, k) model was introduced to analyze tasks, the model guarantees that in a window of k consecutive jobs, at least m jobs must meet their deadline [19].

In [8], the (m, k) model is extended to four definitions for a weakly hard task τ :

1. $\tau \vdash (\binom{n}{m})$ for $1 \leq n < m$: for each set of m consecutive deadlines, τ meets at least n .
2. $\tau \vdash \langle \binom{n}{m} \rangle$ for $1 \leq n < m$: for each set of m consecutive deadlines, τ meets consecutively n of them.
3. $\tau \vdash \overline{\binom{n}{m}}$ for $1 \leq n < m$: for each set of m consecutive deadlines, τ meets at most n of them.
4. $\tau \vdash \overline{\langle n \rangle}$: τ can miss at most n consecutive deadlines.

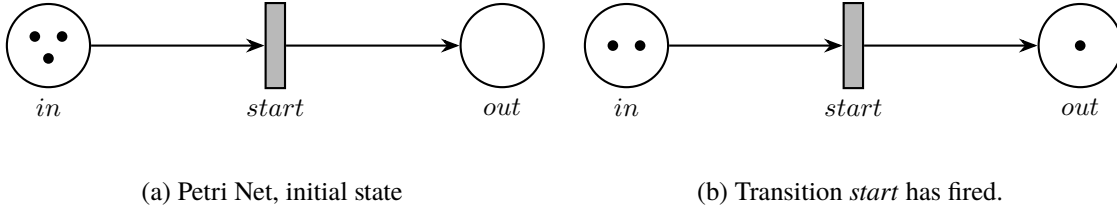


Figure 2.2: Petri Net model, before and after firing

The use of (m, k) model on network scheduling has been explored in [15]. The use of a dynamic priority mechanism to elevate the priority of delayed frames helps enforce (m, k) firm guarantees on the network. This maps the WHRT guarantees to network scheduling policies.

2.3 Petri Nets

The classical Petri Net is a directed bipartite graph with nodes called *places* $\mathcal{P}$ and *transitions* $\mathcal{T}$. These nodes are connected with directed *arcs*. The places are represented by circles and the transitions by bars. Places contain one or more tokens. A place p is called *input place* if a directed arc starts from p to a transition t , p is called *output place* if the directed arc ends in p . A transition is *enabled* if all the input places of the transition have at least one token available for consumption. An enabled transition *fires* by consuming tokens from the input places and producing tokens at the output places [40].

Figure 2.2 depicts a firing sequence for a Petri Net Model. Figure 2.2a shows the initial state of the Petri Net, it consists of two places *in* and *out*, and a single transition *start*. The initial state represents three tokens in place *in*. Figure 2.2b shows the Petri Net after the transition *start* has fired once. The new state is represented by consuming one token from *in* and producing a token in *out*.

The classical Petri Net is not suitable for the modeling in complex large logistics, production, and other systems [45]. Additional extensions were introduced to Petri Net to improve the modeling power. One addition includes *colored tokens*, first discussed in [48], these colored tokens are used to represent information. They can describe simple representational information like a class of jobs or the type of message in a communication system [51]. Jensen and Kristensen [25] defined *colored token* as, the data value attached to a token. The set of possible token colors is defined by means of a type, called color set.

Another extension is the addition of temporal properties to the Net. This helps us measure how the system performs operations, and model and validate real-time systems [36]. Many authors have introduced different notions of how time is applied on the Petri Net. In Zuberek [50], the time is introduced on the transition firings. A firing of a transition occurs in real time, where

tokens are consumed from the input places at the beginning of the firing period and produced at the output places at the end of the firing period, inducing the delay.

In p-timed nets [2], time is introduced as a notion of when the transition should fire. The fire time determines the period of time tokens remain in the input place before transition can be triggered. The firing itself is atomic in this model. A similar notion was introduced by Van Hee et al [21], the timestamp is attached to every token. The timestamp indicates the time the token becomes active in that place. A transition fires when all the input places have an active token.

Definition 2.1 (Jensen & Kristensen [26])

A Timed-Colored Petri Net is a nine-tuple $\mathcal{N}_{tcpn} = (\mathcal{P}, \mathcal{T}, A, \Sigma, V, C, \mathcal{G}, \mathcal{E}, \mathcal{J})$ where:

1. $\mathcal{P}$ is a finite set of places.
2. $\mathcal{T}$ is a finite set of transitions such that $\mathcal{P} \cap \mathcal{T} = \emptyset$.
3. $A \subset (\mathcal{P} \times \mathcal{T}) \cup (\mathcal{T} \times \mathcal{P})$ is a set of directed arcs.
4. Σ is a finite set of non-empty color set. Each color set is either timed or untimed.
5. V is a set of typed variables such that $Type[v] \in \Sigma$ for all $v \in V$.
6. $C : \mathcal{P} \rightarrow \Sigma$ is a color set function that assigns a color set to each place. A place p is timed if $C(p)$ is timed, otherwise p is untimed.
7. $\mathcal{G} : \mathcal{T} \rightarrow EXP_{R_v}$ is a guard function that assigns a guard for each transition t , such that the $Type[\mathcal{G}(t)] = Bool$.
8. $\mathcal{E} : A \rightarrow EXP_{R_v}$ is an arc expression that assigns an arc expression to each arc a , such that $Type[\mathcal{E}(t)] = C(p)$ here p is the place connected to arc a .
9. $\mathcal{J} : \mathcal{P} \rightarrow EXP_{R_\emptyset}$ is an initialization function that assigns an initialization expression to each place p , such that $Type[\mathcal{J}(p)] = C(p)$

For a given $\mathcal{N}_{tcpn}$, the marking function $\mathcal{M}$ maps each place $p \in \mathcal{P}$ to a multiset of colored tokens. The term EXP_{R_v} represents a logical term whose value is evaluated during the execution of the TCPN.

Figure 2.3 extends the classical Petri Net to represent the TCPN model with the colored sets $\Sigma = \{\{black\}\}$, which consists of a single set with the color black. Figure 2.3a represents the initial state of the TCPN, with the *in* place initialized with 3 tokens of color black, but each token is enabled at a different time. The token is represented as $1 \setminus \langle black \rangle @0$, meaning there is one token of color black enabled when the global clock of the model $t = 0$. When the simulation starts at time $t = 0$, only one token in the place *in* is enabled. This triggers the transition *start* to consume the token and produce another token of the same color with timestamp 5 in the place *out*. For time $t = 2 \rightarrow 4$, the transition *start* is never fired because no tokens are enabled. At $t = 5$,

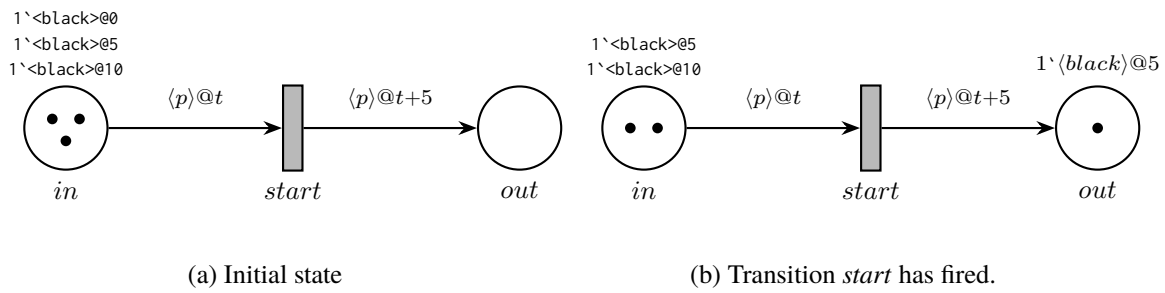


Figure 2.3: TCPN model, before and after firing

the token $1' \langle black \rangle @5$ is consumed from *in*, and the token from the previous transition in *out* is active.

3 Related Works

This chapter explores existing research work that aligns with the thesis. In Section 3.1, we explore the existing research on the stability of NCS, and in Section 3.2 we discuss the existing validation frameworks in NCS.

3.1 Stability of NCS

The stability of NCS under imperfect communications has been studied previously. Zhang et al. [49] derives a stability region plot for the NCS to study the relationship between sampling rate and the network induced delay. Hespanha et al. [23] survey the stability results across a range of closed-loop models in the presence of network sampling, delays and packet dropout. Schenato et al [42] established fundamental limits of control over lossy networks, and implies the need for a joint design of control and network. Recently, WHRT models [19] [8] have been used to analyze the stability of NCS. Blind and Allgöwer [9] were able to guarantee Lyapunov stability of the closed-loop system using WHRT constraints. Further, Maggio et al. [37] provides a closed-loop stability condition for LTI systems, bridging the (m, k) firm model to spectral radius stability criteria. These works mostly focus on the system stability under a fixed operating condition and derive an upper bound on the tolerance to network delay. However, these works do not consider the change in operation and the stability violations that can occur during the transition window.

There has been substantial work on the stability of system that switches between multiple controllers. Liberzon and Morse [35] studied the fundamental stability problem in such switched systems and defined dwell-time as a stability constraint. Liberzon [34] further analyzes the stability for different models of the switched system. Hespanha and Morse [22] introduced the concept of average dwell-time to allow occasional fast-switch of control signals. These literatures focus switch entirely on the control signal, the switching is between control gains, and the switch is considered to be instantaneous. The network layer is not modeled and the time required to reconfigure the network is not considered.

On the network side in NCS, IEEE 802.1 Time-Sensitive Networking (TSN) introduces a standard that provides real-time guarantees in Ethernet networks. The Time-Aware Shaper from TSN supports the latency and reliability requirements of the streams in the closed-loop system[24]. Egger et al. [14] provides the formal end-to-end guarantees for streams in wireless TSN by

introducing Full Interleaving Packet Scheduling (FIPS) as a wireless-friendly IEEE 802.1Qbv scheduler. Further, Egger et al. [15] introduced the (m, k) firm elevation policy for 5G TSN, enforcing delivery guarantees when network conditions deviate from normal. Additionally, Ahrendts et al. [1] developed methods to verify the weakly hard constraints of traffic streams in the context of switched networks. These works provide different levels of guarantees for the network schedule to ensure the delivery of packets across the network. However, they do not address the relationship between the network guarantees and control stability, particularly during the transition window when the schedule reconfigures.

Baumann et al.[7] demonstrated stability in fast feedback loops over multi-hop wireless networks while changing the operating condition. However, the paper focuses on closed-loop formal guarantee in LTI for their synchronized mode change protocol, which ensures deterministic delay for model change, rather than a framework for validating the co-design of control and network during mode transition.

In this thesis, we bridge the existing research by defining the *mode of operation* as a property of both the plant and the network schedule. We formalize a worst-case transition time that captures the network guarantees and the delay incurred from reconfiguring schedules, and derive a sufficient condition that maps to the control-side tolerance to stability. This enables us to analyze the stability of NCS during mode switch using the parameters from both sides of the domain.

3.2 Validation Frameworks for NCS

In this section, we explore the existing frameworks for NCS. Cervin et al.[10] developed two frameworks called *JitterBug* and *TrueTime* for NCS analysis. *JitterBug* enables us to assert the sensitivity of the control loop to sampling, delays, and other timing problems. The sensitivity is measured as a quadratic performance criterion. It allows the analysis of a number of continuous and discrete linear systems, as well as a stochastic model to simulate the execution of the system. *TrueTime* is a co-simulation of controller tasks and the continuous dynamics of plants. The simulator captures the timely behavior of controller tasks and communication networks by evaluating performance from a control perspective. In Tamssaouet and Amari [44], the NCS was modeled using timed-automata [3] to formally verify the upper bound on response time. These frameworks share a common limitation: they model the network as a fixed stochastic distribution and do not allow the modeling of different modes and the representation of a transition window. Furthermore, the existing tools analyze performance metrics such as quadratic cost and average delay rather than formal delivery guarantees, such as WHRT guarantees.

In this work, we explore Petri Nets [40] for our framework. Jensen and Kristensen [26] [28] formally define TCPN and provides a framework for simulation and performance analysis of concurrent systems. Examples of the application of Petri Nets in the industry can be found in [41]. Formal verification properties, such as liveness and reachability analysis, are not explored in this

work. Instead, the Petri Net is modeled as a simulation tool to produce the delivery of packets in NCS, and to validate the traces.

4 System Model and Problem Statement

In this chapter, the system model is introduced to represent the NCS. Section 4.1 defines the system topology, plant dynamics, and modes of operation, and Section 4.2 defines the problem we address in this thesis.

4.1 System Model

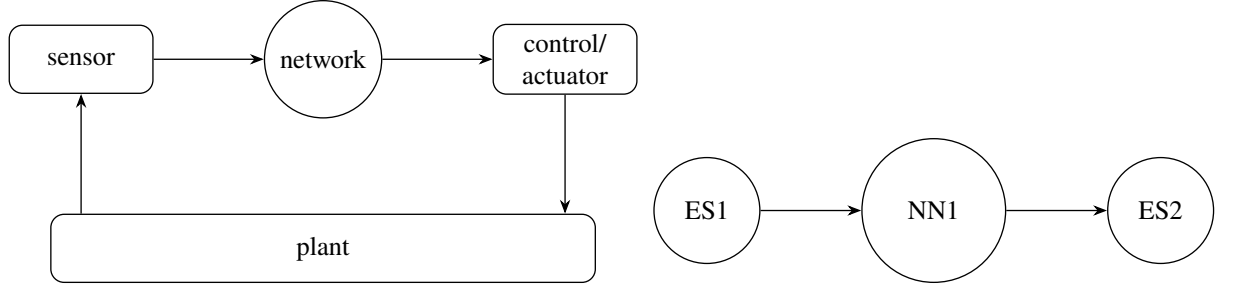
The NCS consists of three components, the sensor, the controller, and the actuator, that form the boundary of the control loop. These components can either be directly connected with each other via point-to-point Ethernet links, or they could be distributed across a shared network with switches, bridges and other network components between them [23]. These network components introduce variable delay and susceptibility to packet loss [49]. We model the system as consisting of two types of nodes: ES components that form the boundary of the control loop, and NN components through which packets of the control loop traverse.

4.1.1 Topology

We define the NCS as a graph $G = (V, E)$, where $V = V_{\text{ES}} \cup V_{\text{NN}}$. An end system $v \in V_{\text{ES}}$ represents the sensor, controller or actuator. Packets for the control loop originate and terminate in nodes of the type ES. A node $v \in V_{\text{NN}}$ represents a network component like the switch or bridge, packets never terminate on the NN nodes. The edges E shows the links between the nodes.

$$\begin{aligned} G &= (V, E) \\ V &= V_{\text{ES}} \cup V_{\text{NN}} \\ E &\subseteq (V_{\text{ES}} \times V_{\text{NN}}) \cup (V_{\text{NN}} \times V_{\text{NN}}) \cup (V_{\text{NN}} \times V_{\text{ES}}) \end{aligned} \tag{4.1}$$

Figure 4.1a shows a NCS architecture with the sensor transmitting sampled data over the network to controller and actuator. Figure 4.1b models the NCS as a graph G .



(a) Networked control system architecture.

(b) The graph G corresponding to the NCS

4.1.2 Deadline Miss and Stability

In a NCS, if a control packet fails to arrive at the actuator within the next sampling instant, we assume it has missed the deadline. Under the ZOH assumption, the actuator continues to control the plant with the last received control update. During this interval, the plant continues to evolve in an open loop. Consecutive deadline miss would have a compounding effect on the open-loop evolution and result in a divergent trajectory.

In Section 2.2, we discussed different models for representing WHRT guarantees. We are interested in the n consecutive *miss* model of representation, $\tau \vdash \langle \bar{n} \rangle$. For stability analysis of a control system under open-loop conditions, this model is sufficient to capture the worst-case of maximum consecutive deadline missed before the system diverges. Other models can also be used to represent system stability, but they limit the stability observation to a window of packets. We adopt the model and represent $\mathcal{S}_{\text{app}}$ the maximum consecutive deadline miss the plant can tolerate for a given gain matrix K and sampling period h .

4.1.3 System Behavior and Modes of Operation

The control gain K value affects how the system reacts to deviations. This means we can design K to make the system perform in the desired way. We define system behavior $\mathcal{B}$ to describe the intended way the system reacts to perturbations, it decides whether the system should correct deviations from equilibrium aggressively or in a more controlled way. The behavior $\mathcal{B}$ determines the value of K . In our LQR control, the behavior of the system can be characterized by the Q and R matrices from Equation (2.5).

In the discrete systems, the same behavior $\mathcal{B}$ can be achieved with different periods for the control loop by varying the gain values. For different operating periods of the control loop, the system has different tolerances to deadline miss. To this end, we define the *mode of operation* π as a tuple:

$$\pi = (\mathcal{B}, h, K) \quad \pi \vdash \langle \mathcal{S}_{\text{app}} \rangle \quad (4.2)$$

The mode π represents how the control loop operates to achieve the desired system behavior $\mathcal{B}$. In Equation (4.2), the mode π shows that when the control loop operates with a period of h and control gain K the system behaves according to $\mathcal{B}$ as long as the network guarantees at most $\mathcal{S}_{\text{app}}$ deadline misses.

4.1.4 Time Triggered Streams

For each mode π_i , there exists a periodic transmission of control packets between sensor and actuator. We model this as time-triggered streams similar to [14], for each mode π_i , there is a stream f that delivers the control packet to the actuator.

$$f = (\text{path}, h, d) \quad (4.3)$$

where $\text{path}(f) = \{v_1, v_2, \dots, v_n\}$ is an ordered path in G with $v_1, v_n \in V_{\text{ES}}$ and $v_j \in V_{\text{ES}} \cup V_{\text{NN}}$ for $1 < j < n$. h is the sampling period of the control loop for the mode π_i , and d is relative deadline.

In this control loop, the sensor-to-controller and controller-to-actuator streams together form f to close the loop. For the system to operate with a 1-step delay as per Equation (2.4), the control loop must close before the next sampling instant. This requires the deadline of stream f in a mode to satisfy $d_f = h$.

A stream f releases a sequence of *packets*. The k_{th} packet released at time $r_k = k \cdot h$ must arrive at $v_n \in \text{path}(f)$ before its absolute deadline $r_k + d_f$. We say the packet is a *hit* if it arrives at v_n before the deadline, and a *miss* if it arrives after.

4.1.5 Network Schedule

The network nodes V_{NN} are responsible for transmitting the packets across G . We assume the nodes to operate under a network schedule τ that determines whether each packet is delivered within its deadline. We represent the worst-case of consecutive packets the schedule cannot guarantee as $\mathcal{S}_{\text{net}}$. A schedule $\tau \vdash \langle \mathcal{S}_{\text{net}} \rangle$, if no more than $\mathcal{S}_{\text{net}}$ consecutive misses occur in the schedule. We represent the pattern of $\mathcal{S}_{\text{net}} = n$ for τ as follows:

$$\{1, 0_1, 0_2 \dots 0_n\} \quad (4.4)$$

Equation (4.4) represents that there are at most n consecutive deadline misses in schedule τ .

4.1.6 Safety Criteria

It has been established in [30] that the trajectories of the nonlinear system are close to the trajectories of the linear system around a small neighborhood around the equilibrium. This indicates that the behavior of the nonlinear system can be extracted from the behavior of the linearized system only around a small operating region of state space. We define that the state of the system is safe if and only if the current state is within the operating region.

The output we are interested in observing is the control output at time t , i.e., $y(t) = Cx(t)$ from Equation (2.2), hence observing the state space of the system. The system safety under mode π_i is defined as follows:

$$\forall t \geq 0 : \|y(t) - y_{\text{equilibrium}}\| \leq \epsilon \iff \sigma(\pi_i) \quad (4.5)$$

where $y_{\text{equilibrium}}$ is the system state with respect to linearization and ϵ is the neighborhood around which the linearization holds. The sufficient condition to guarantee a system operating in a single mode π_i can be stated as:

$$\tau \vdash \langle S_{\text{net}} \rangle \wedge S_{\text{net}} \leq S_{\text{app}} \implies \sigma(\pi_i) \quad (4.6)$$

4.1.7 Assumptions

The following are the assumptions for our system model:

- A1: LTI plant dynamics, the system state space is linearized around one or more dimensions of the state.
- A2: Full-state feedback, the sensor can sense the complete state vector x , and the control has access to complete state.
- A3: In between two control packets, the plant evolves under a constant input u derived from the last received control packet.
- A4: Negligible computation delay, we assume that the computation time required to computing $-K \cdot x[k]$ is negligible.
- A5: Synchronized clocks for all nodes in G .
- A6: Single control loop, we assume the system has a single control loop, sensor-to-controller, and controller-to-actuator.

4.2 Problem Statement

In Section 4.1.6, we established the stability criteria for the system while operating in a fixed mode. In practice, these systems exhibit more than one behavior or exhibit the need to operate under different conditions while exhibiting the same behavior. For example, an Automated Guided Vehicle (AGV) on a factory shop floor might have two behaviors, one for speed maintenance to move objects and another for distance maintenance for parking. Hence, there is a need for the system to adapt the operating condition in response to changes in events from the environment or within the system.

Baumann et al. [7] provide a formal stability guarantee for LTI for the synchronized mode change protocol. The situation when the control or network prematurely switches mode has not been investigated. We investigate the upper bound on stability during mode transition for a system in open-loop and define a sufficient condition for the safety of these systems.

To address this, we first formally define mode sequence and a sufficient condition for safety in Section 5.1 and then develop a validation framework using TCPN that models the behavior of the stream, schedules, mode switch, and reconfiguration delay and reports the safety of the system with respect to Equation (4.6) and Equation (5.6). The framework takes as input (G, f, δ) and reports the packet trace observed at the ESs. This helps us validate the following:

1. Validate the sufficient conditions, Equation (4.6), Equation (5.6), for a given configuration.
2. Explore the trade-off between reconfiguration delay, $\mathcal{S}_{\text{net}}$, and their effect on safety of the system.
3. Identify the boundary until which the system safety holds beyond the sufficient conditions

We evaluate this framework on an inverted pendulum on a cart control system. The system operates over a shared network and has predefined behaviors. We validate the condition across different δ for the reconfiguration. These results are then validated against the pendulum simulator to assess the soundness of our conditions.

5 Framework Design

In Chapter 4, we defined the system model, mode of operation, and the sufficient conditions for the system to remain safe in that mode. In this chapter, we formally define mode switch and derive a sufficient condition for system safety during mode transition in Section 5.1. We introduce the formal mapping from the system model using places, timed sets, initial markings, and transitions in Section 5.2. In Section 5.3, we discuss how the mode switch is simulated in the Petri Nets. Section 5.4 examines how the tokens are observed for reporting and Section 5.5 defines how the validation of the sufficient conditions from Equation (4.6) and Equation (5.6) is computed in the framework. Section 5.6 briefly discusses the implementation of the framework, and Section 5.7 shows an example of the modeling of TCPN and its simulation.

5.1 Mode Switching Model

In this section, we define the ability to modify the operating condition as a *mode switch* and the sequence of modes through which the system switches during its lifetime as a *mode switch sequence*, i.e.,

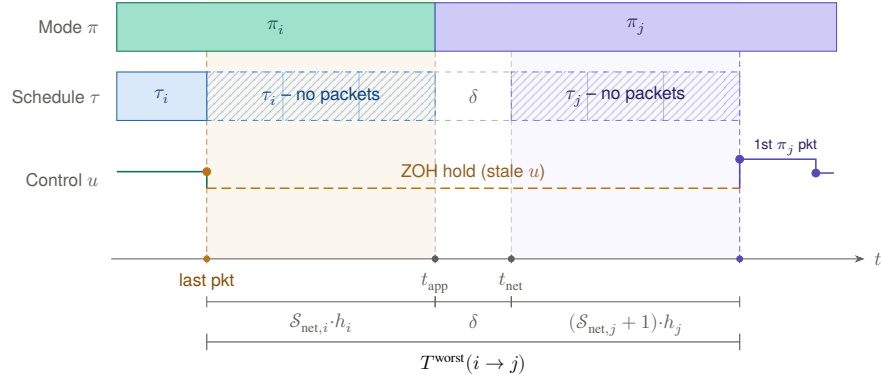
$$\Pi = \langle (\pi_1, t_1), (\pi_2, t_2), \dots \rangle \quad (5.1)$$

Each pair (π_i, t_i) represents the mode π_i and the time t_i at which the system starts operating in that mode. Equation (4.6) establishes the sufficient condition for the system to be safe in that mode. However, mode transition is not instantaneous, and delay must be considered before the operating mode π_i and the network schedule are compatible.

Application Initiated Switch: The system requests a mode change from π_i to π_j at the application level when its operating goal changes. The system switch from K_i, h_i to K_j, h_j and the network reconfigures from τ_i to τ_j . If the application switches at time t_{app} , then the network reconfigures at a time t_{net} :

$$t_{\text{net}} = t_{\text{app}} + \delta_{\text{net-reconfig}} \quad (5.2)$$

During the time $[t_{\text{app}}, t_{\text{net}})$, the application operates under mode π_j and the network operates under a schedule τ_i that might no longer guarantee the safety of the system.


 Figure 5.1: Transition window considering the worst-case of *miss* from network

Network Initiated Switch: The network changes the schedule from τ_i to τ_j due to resource contention and can no longer operate under the current schedule. The new schedule might be more relaxed with a higher S_{net} than τ_i . The application adapts at a time:

$$t_{\text{app}} = t_{\text{net}} + \delta_{\text{app-reconfig}} \quad (5.3)$$

During the time $[t_{\text{net}}, t_{\text{app}})$, the application operates under π_i and network uses the new schedule τ_j . The current schedule might have an S_{net} higher than S_{app} , which might not be compatible with the application mode. In this work, we focus on *Application Initiated Switch*, where the plant mode transitions first and then network reconfigures after a delay δ .

5.1.1 Sufficient Condition for Mode Switch

During the transition window, the safety of the system can no longer be guaranteed as the system evolves in open loop under ZOH. Hence, it is essential to ensure that the network reconfiguration delay δ doesn't exceed upper bound on the deadline miss.

$$\delta \leq \mathcal{S}_{\text{active}} \cdot h_{\text{active}} \quad (5.4)$$

where $\mathcal{S}_{\text{active}} \cdot h_{\text{active}}$ represents the parameters of the mode active during the transition.

Equation (5.4) captures the best-case switching condition, where the switch is initiated immediately after the plant has received a *hit* packet and receives the first packet of the new mode without any *miss*. It is not necessary that the system switches exactly after a *hit* and immediately receives the packet in new mode. The worst-case time for which the system remains in open loop, shown in Figure 5.1, can then be modeled by assuming that the transition is initiated after the $\mathcal{S}_{\text{net}, i}$ misses and the packet from the new mode is received after the $\mathcal{S}_{\text{net}, j}$ misses, where i and j represent

the index of the mode π and schedule τ . The end-to-end worst-case time can be represented as follows:

$$T_{i \rightarrow j}^{\text{worst}} = \mathcal{S}_{\text{net}, i} \cdot h_i + \delta + (\mathcal{S}_{\text{net}, j} + 1) \cdot h_j \quad (5.5)$$

The system can be guaranteed to be safe during $T_{i \rightarrow j}^{\text{worst}}$ only if the following condition holds:

$$T_{i \rightarrow j}^{\text{worst}} \leq \min\{\mathcal{S}_{\text{app}, i} \cdot h_i, \mathcal{S}_{\text{app}, j} \cdot h_j\} \quad (5.6)$$

For any given mode sequence Π , the system can be guaranteed to be safe if Equation (5.6) holds for every consecutive pair $(\pi_i, t_i), (\pi_j, t_j)$ in Π . This sufficient condition helps us model the δ as a design parameter, reducing δ helps us relax the $\mathcal{S}_{\text{net}}$ for the network.

5.2 Formal Model: System Model to TCPN

The framework operates as a pipeline, the designers of the NCS model the system using the semantics of Chapter 4 and Section 5.1 as JSON. The framework then transforms the JSON into a TCPN model and simulates the timed behavior of the packet in the control loop and reports the observed delivery guarantee. Figure 5.2 shows the stages of the pipeline.

The TCPN has a concept of time that enables the net to capture the time taken by events in the system [27]. This allows us to encode the schedule, stream period, and reconfiguration delay as a timed constraint within the net. During the mode switch, TCPN can capture the transition window and report the packet trace.

The framework should model the following properties of the system:

- The streams from Section 4.1.4.
- The network schedule τ as a cyclic pattern of *hit* or *miss*.

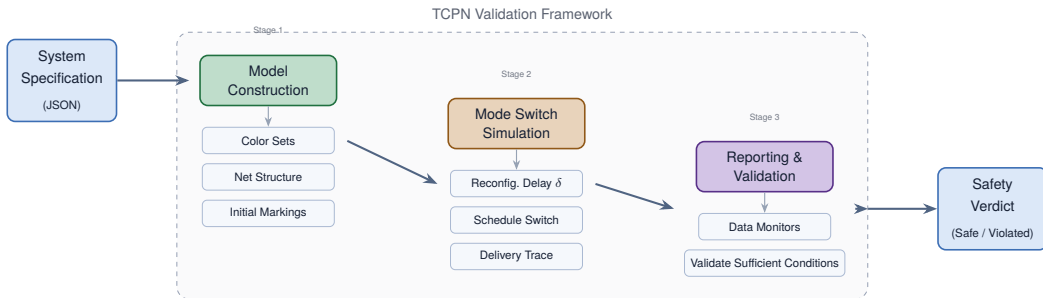


Figure 5.2: TCPN framework pipeline

- The packet for streams depending on modes and cyclic pattern depending on network mode.
- Report the delivery guarantees across the run time of the system.

In this section, we discuss how the system model defined previously maps to the formal terms of $\mathcal{N}_{\text{tcpn}}$, where $\mathcal{N}_{\text{tcpn}} = (\mathcal{P}, \mathcal{T}, A, \Sigma, V, C, \mathcal{G}, \mathcal{E}, \mathcal{J})$ from Definition 2.1.

5.2.1 Colored Sets

We define three colored sets that define the corresponding data type flowing through the $\mathcal{N}_{\text{tcpn}}$. For a known system model, we define the colored sets as follows:

Σ_{mode} represents the mode of operation π from Equation (4.2). It is used to show the active mode associated with a stream or schedule, i.e.,

$$\Sigma_{\text{mode}} = \{1, 2, \dots, n\} \quad (5.7)$$

where each value i references the mode π_i in the mode sequence Π . This colored set is used in both stream and network places. At each level they indicate the mode active in that level.

Σ_{stream} represents the stream configuration f . We define the colored set as the exhaustive list of stream under consideration in the system, i.e.,

$$\Sigma_{\text{stream}} = \{1, 2, \dots, m\} \quad (5.8)$$

where each value j references stream f_j .

Σ_{pkt} represents the colored set of packets. Let $\text{ID} = \mathbb{N}$ denote the packet sequence number, then the color set is as follows:

$$\Sigma_{\text{pkt}} = \text{ID} \times \Sigma_{\text{stream}} \times \Sigma_{\text{mode}} \quad (5.9)$$

The packet type contains a triple where the first element is the unique identifier of the packet, the second refers to the stream and the third refers the mode. Alternatively, the packet token (k, j, i) represents the k^{th} packet generated by stream f_j under the application mode π_i .

Additional Functions

The Σ_{mode} and Σ_{stream} are integer indices, the properties of these modes and streams are defined through set of relations over these indices. These relations can be used to define *guards* and *arc* expressions.

Mode Mapping

$$streams : \Sigma_{\text{mode}} \rightarrow f \quad (5.10)$$

$$schedule : \Sigma_{\text{mode}} \rightarrow \tau \quad (5.11)$$

where $streams(i)$ maps to stream f_i active in π_i . Similarly, $schedule(i)$ maps to network schedule τ_i designed for π_i .

Stream Mappings We define $src(j)$, $dst(j)$, $period(j)$, and $deadline(j)$ maps to the source node, destination node in V_{ES} , the period and deadline respectively for the stream f_j .

The three color sets define the Σ for $\mathcal{N}_{\text{tcpn}}$, where $\Sigma = \{\Sigma_{\text{mode}}, \Sigma_{\text{stream}}, \Sigma_{\text{pkt}}\}$.

5.2.2 Net Structure

Places

The places in our $\mathcal{N}_{\text{tcpn}}$ are partitioned into two sets. One, includes the places that are related to the nodes in G , called network places $\mathcal{P}_{\text{net}}$. The other set called $\mathcal{P}_{\text{stream}}$, is derived from the stream f . For each place $\mathcal{P}$, we define $C(\mathcal{P})$ as the type of the tokens in that place, the function maps to one of the color sets defined in Section 5.2.1. All the places in $\mathcal{P}$ are timed places, i.e., the tokens have a timestamp to it.

Network Places: for each node $v \in V$ there exists a place in $\mathcal{P}_{\text{net}}$. Additionally, $\forall v \in V_{\text{NN}}$ there exists a place to represent the mode of the network node. Hence, the places in $\mathcal{P}_{\text{net}}$ can be divided into:

- p_i^{es} represents a place for the end system node, where $v_i \in V_{\text{ES}}$. The color type for the place is defined as $C(p_i^{es}) = \Sigma_{\text{pkt}}$.
- p_i^{nn} represents the buffer in the network node, where $v_i \in V_{\text{NN}}$. The color type for the place is defined as $C(p_i^{nn}) = \Sigma_{\text{pkt}}$.
- p_i^{mode} represents a place that determines the mode of the network node and in turn the schedule to be used on the buffer, where $v_i \in V_{\text{NN}}$. The color type for the place is defined as $C(p_i^{mode}) = \Sigma_{\text{mode}}$.

Stream Places: for each periodic stream f_i are represented in Table 5.1.

Place	Description	Color Set $C(p)$
p_j^{mode}	Active mode of the stream	Σ_{mode}
p_j^{pkt}	Next packet to be generated	Σ_{pkt}
p_j^{str}	Stream configuration	Σ_{stream}
p_j^{done}	Delivered packets	Σ_{pkt}

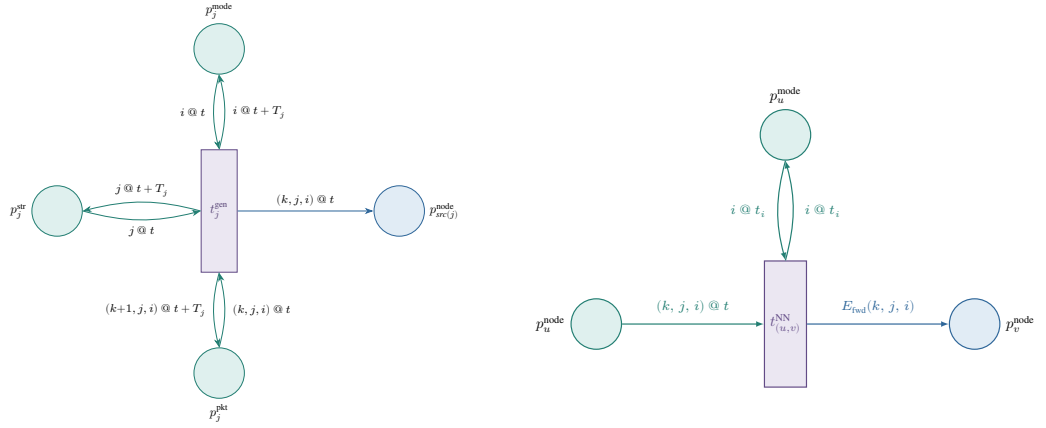
Table 5.1: Stream places in $\mathcal{P}_{\text{stream}}$ and their associated color sets.(a) Stream generating transition, where $T_j = \text{period}(j)$ (b) Network schedule transition $t_{(u,v)}^{\text{NN}}$.

Figure 5.3: TCPN transitions for periodic stream generation and network scheduling.

5.2.3 Transitions, Arcs and Arc Expressions

In this section we define the transition nodes $\mathcal{T}$, their arcs A and arc expression function $\mathcal{E}(a)$ defined in Definition 2.1 for our $\mathcal{N}_{\text{tcpn}}$. For each transition we specify the input and output arcs, and the expression that governs the token transfer.

Periodic Packet Generation: For each periodic stream f_j , the transition generates the next packet for the stream f_j and deposits in the $\text{src}(j)$. The transition is represented as t_j^{gen} and their arcs and expressions are expressed below:

Input arcs	E(a)	Output arcs	E(a)
$p_j^{\text{mode}} \rightarrow t_j^{\text{gen}}$	$i@t$	$t_j^{\text{gen}} \rightarrow p_j^{\text{mode}}$	$i@t + \text{period}(j)$
$p_j^{\text{stream}} \rightarrow t_j^{\text{gen}}$	$j@t$	$t_j^{\text{gen}} \rightarrow p_j^{\text{stream}}$	$j@t + \text{period}(j)$
$p_j^{\text{packet}} \rightarrow t_j^{\text{gen}}$	$(k, j, i)@t$	$t_j^{\text{gen}} \rightarrow p_j^{\text{packet}}$	$(k + 1, j, i)@t + \text{period}(j)$
		$t_j^{\text{gen}} \rightarrow p_{\text{src}(j)}^{\text{node}}$	$(k, j, i)@t$

Figure 5.3a represents the periodic packet transition and its arcs. The transition first consumes the token from the mode, stream and packet place, represented by the input arcs. It then returns the mode, stream and the packet token with the next sequence number to their respective places with a delay of $period(j)$, scheduling the next cycle of packet generation. The current packet is placed in the source end system of the stream f_j .

Routing transitions: For each edge (u, v) in G , we define the following transitions that are responsible for transmitting packets from one place to another. Depending on the place type these transitions are of two types:

- if $u \in V_{NN}$, the transition $t_{(u,v)}^{NN}$ acts as the egress queue of the network node. The transition consumes the mode of the network p_u^{mode} and the packet token from p_u^{NN} , and deposits the packet token into p_v . Let us denote the $sched(i)$ as the worst-case consecutive *miss* pattern after a successful *hit* for the schedule τ_i compatible for the mode i . We define a cyclic counter c_{i,f_j} to return the current index of $sched(i)$ for the mode i and stream f_j .

Input arcs	E(a)	Output arcs	E(a)
$p_u^{mode} \rightarrow t_{(u,v)}^{NN}$	$i@t_i$	$t_{(u,v)}^{NN} \rightarrow p_u^{mode}$	$i@t_i$
$p_u^{node} \rightarrow t_{(u,v)}^{NN}$	$(k, j, i)@t$	$t_{(u,v)}^{NN} \rightarrow p_v^{node}$	$E_{fwd}(k, j, i)$

$$E_{fwd}(k, j, i) = \begin{cases} (k, j, i), @, t + d_{hit} & \text{if } sched[c_{(i,j)}] \bmod |\tau| = 1 \\ \emptyset & \text{if } sched[c_{(i,j)}] \bmod |\tau| = 0 \end{cases} \quad (5.12)$$

Figure 5.3b represents the transition to transmit the packet through the network, the arc expression evaluates the function Equation (5.12), the function checks for a given mode of schedule and stream if the current packet in the network node is a *hit* or *miss*. On a *hit*, the packet is forwarded to the next place with a delay d_{hit} . On a *miss*, no token is produced. In this way, the transition simulates the behavior of the network node.

- if $u \in V_{ES}$ then for every stream f_j with source as u , we define transition t_j^{es} that consumes a packet of stream f_j from u and places it in the corresponding place associated with v .

Input arcs	E(a)	Output arcs	E(a)
$p_u^{node} \rightarrow t_j^{es}$	$(k, j, i)@t$	$t_j^{es} \rightarrow p_v^{node}$	$(k, j, i)@t + d(u, v)$

where $d(u, v)$ is the link delay. The packet is forwarded with it a delay $d(u, v)$. The transition and arcs are represented in Figure 5.4a.

Delivery transitions: for each stream f_j we define a transition t_j^{done} responsible for observing the packets reaching the destination node for each stream and reporting. Consumes from $p_{dst(j)}^{es}$ and produces into p_j^{done} as depicted in Figure 5.4b.

5 Framework Design

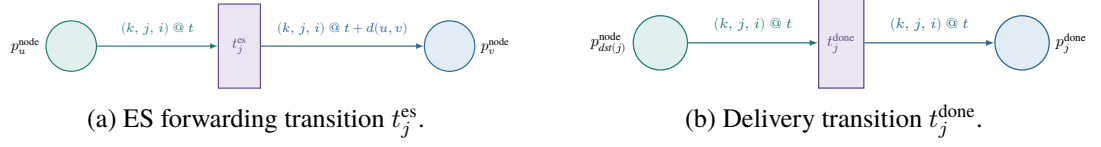


Figure 5.4: TCPN transitions for end system forwarding and packet delivery.

Input arcs	E(a)	Output arcs	E(a)
$p_{dst(j)}^{\text{node}} \rightarrow t_j^{\text{done}}$	$(k, j, i) @ t$	$t_j^{\text{done}} \rightarrow p_j^{\text{done}}$	$(k, j, i) @ t$

The packet is moved from the stream's destination node to the stream's done buffer for reporting.

5.2.4 Guards

Guards are defined for transitions whose input might have packets from more than one stream. These guards should return *true* for the transition to trigger.

Guard for Delivery Transitions:

$$\mathcal{G}(t_j^{\text{done}})(f_{\text{id}}) = \begin{cases} \text{true} & \text{if } f_{\text{id}} = j \\ \text{false} & \text{otherwise} \end{cases} \quad (5.13)$$

The $dst(j)$ can be the destination node of other streams as well. Hence, it is important that the delivery transition observes packet from stream f_j . All other transitions in $\mathcal{N}_{\text{tcpn}}$ have a guard expression that evaluates to true, $G(t) = \text{true}$.

5.2.5 Initial Markings

The Initialization function $\mathcal{I}$ defines the set of token in the Petri Net at $t = 0$. $\mathcal{I}$ depends on the first pair π_i, t in the mode sequence Π . Let us assume that the initial mode of the system to π_i . **Stream Places:** for each stream periodic stream f_j where and $j \in \text{streams}(i)$.

$$\mathcal{I}(p_j^{\text{mode}}) = 1 \setminus i @ t \quad (5.14)$$

$$\mathcal{I}(p_j^{\text{stream}}) = 1 \setminus j @ t \quad (5.15)$$

$$\mathcal{I}(p_j^{\text{packet}}) = 1 \setminus (1, j, i) @ t \quad (5.16)$$

The first packet has a sequence number $k = 1$. The mode and stream tokens are available from time t_i , the release time, enabling the firing of t_j^{gen} at t_i . The prefix 1 in the expression represents the number of tokens initialized.

Network Places: for all nodes $u \in V_{\text{NN}}$ we initialize a mode token for all the network nodes as follows:

$$\mathcal{J}(p_v^{\text{mode}}) = 1 \cdot i @ t \quad (5.17)$$

This ensures that all network node transmits packets using the schedule $\text{sched}(i_0)$.

For all other places, the initial marking is an empty set.

$$\mathcal{J}(p) = \emptyset \quad \text{for all remaining } p \in P \quad (5.18)$$

This includes all ES places, done places, and stream place for streams not active in mode π_i . Tokens enter these places when the transition fires during simulation.

5.3 Mode Switch Simulation

In this section, we introduce the mode switch simulation design.

5.3.1 Application Mode Switch Time

The application reconfiguration time t_{app} represents the time at which the application switch mode from π_i to the next mode π_j in the mode sequence Π . We categorize the t_{app} based on the open loop state of the system into the following:

- **Best-Case Switch Time:** The mode switch occurs at an instant immediately followed by a *hit* of a control packet. This means the plant has just received the control update when the mode switch occurs.
- **Worst-Case Switch Time:** The mode switch occurs at an instant when the system has experienced $\mathcal{S}_{\text{net}, i}$ misses. This means that the plant has evolved in open loop for $\mathcal{S}_{\text{net}, i} \cdot h_i$ before the application switches mode.

Similarly, once the network has reconfigured to the schedule τ_j , the first packet under the new mode can be subjected to the above two scenarios. Best-case, the system receives the first control packet in new mode. Worst-case, the system receives the packet after $\mathcal{S}_{\text{net}, j}$ misses, increasing the open loop time until $\mathcal{S}_{\text{net}, j} \cdot h_j$.

For the framework to validate the sufficient condition in Equation (5.4) and Equation (5.6) for the *best-case switch time* and *worst-case switch time*, we parametrize the number of misses before and after switch the framework must simulate to express this behavior. We define $\hat{\mathcal{S}}_i$ and $\hat{\mathcal{S}}_j$ to represent the number of *miss* the framework waits before and after the switch.

$$0 \leq \hat{\mathcal{S}}_i \leq \mathcal{S}_{\text{net},i} \quad (5.19)$$

$$0 \leq \hat{\mathcal{S}}_j \leq \mathcal{S}_{\text{net},j} \quad (5.20)$$

The framework realizes the above parameters by observing the cyclic counters of the network schedule. Let us consider the mode switch from $\pi_i \rightarrow \pi_j$ occurs at t_i in the sequence Π . When the simulation clock reaches t_i , the framework prepares for a mode switch by inspecting the current index c of the cyclic counter. If the c represents a position that translates to $\hat{\mathcal{S}}_i$, the mode switch is triggered immediately. Else, the framework waits until the c represents the position that ensures $\hat{\mathcal{S}}_i$ misses have occurred.

$$t_{\text{app}} = t_i + w \cdot h_i, \quad w = (\hat{\mathcal{S}}_i - c) \quad (5.21)$$

The network reconfigures at $t_{\text{net}} = t_{\text{app}} + \delta$. When the network is reconfigured to use mode π_j schedule, it initializes the schedule to begin such a way that it simulates the $\hat{\mathcal{S}}_j$ misses before the first control packet in the new mode is received.

5.3.2 Markings Switch

In TCPN, the mode switch is executed in following phases. We continue the assumption from above that states the mode switch is from $\pi_i \rightarrow \pi_j$.

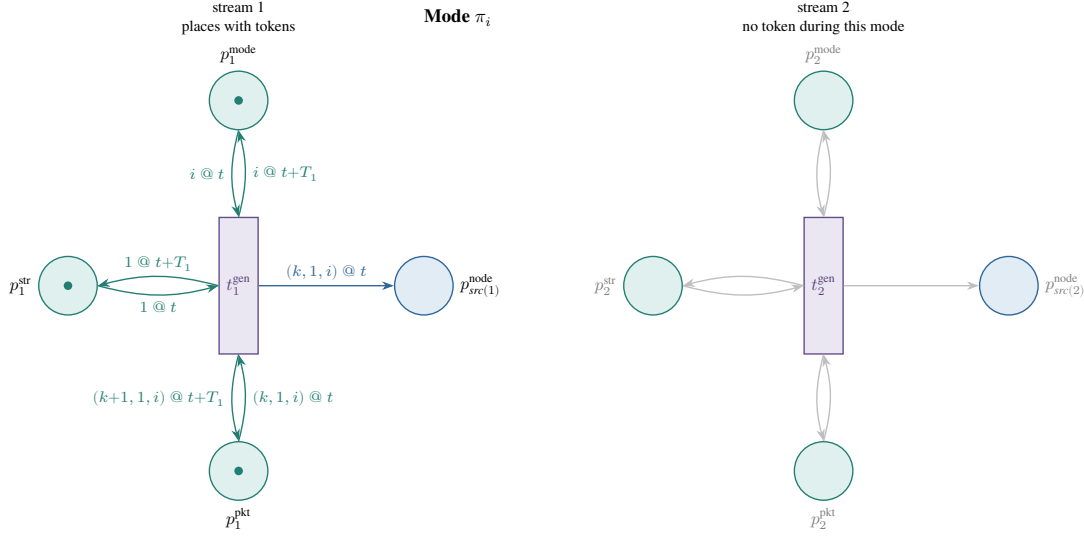
Application reconfiguration at t_{app}

- Clear *markings* from all stream places that are active in the mode π_i . For every periodic stream $f_i \in \text{streams}(i)$, the corresponding stream, mode and packet token are removed.

$$\mathcal{M}(p_i^{\text{mode}}) = \emptyset \quad \mathcal{M}(p_i^{\text{pkt}}) = \emptyset \quad \mathcal{M}(p_i^{\text{str}}) = \emptyset \quad (5.22)$$

- Set the tokens for the stream places in mode π_j . For every periodic stream $f_k \in \text{streams}(j)$, the corresponding stream, mode and packet tokens are set.

$$\mathcal{M}(p_k^{\text{mode}}) = 1 \setminus j @ t_{\text{app}} \quad \mathcal{M}(p_k^{\text{pkt}}) = 1 \setminus k @ t'_{\text{app}}, \quad \mathcal{M}(p_k^{\text{str}}) = 1 \setminus \langle 1, j, k \rangle \quad (5.23)$$


 Figure 5.5: Mode π_i : stream 1 active, stream 2 inactive.

Network reconfiguration at t_{net} .

- Replace the mode token at the mode place for all network node.

$$\forall v \in V_{\text{NN}} : \mathcal{M}(p_v^{\text{mode}}) = 1 \setminus j @ t_{\text{net}} \quad (5.24)$$

- The cyclic counter for the schedule τ_j is set to simulate the $\hat{\mathcal{S}}_j$ misses before the *hit* is simulated.

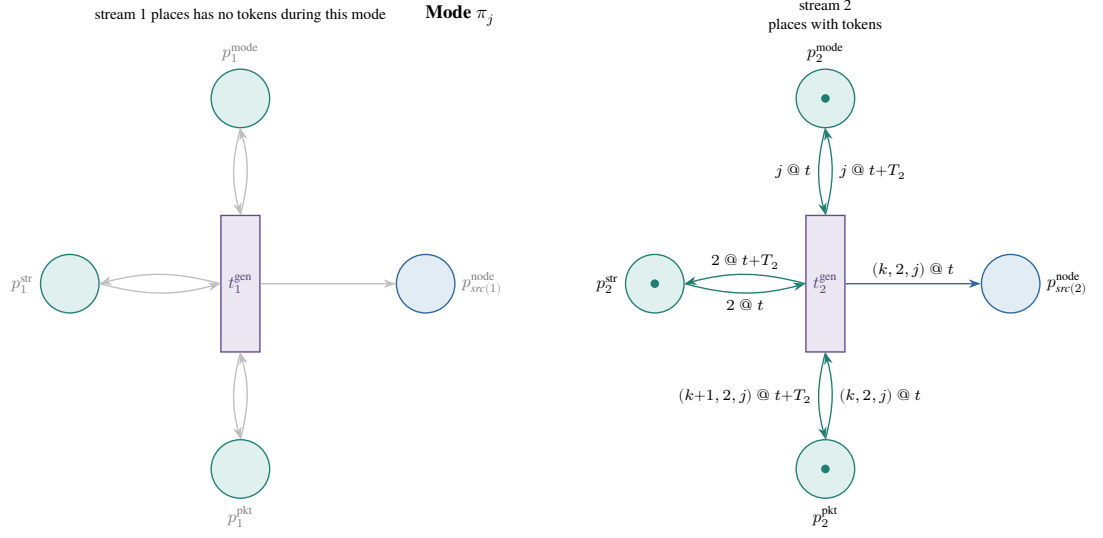
The parameters $\hat{\mathcal{S}}_i$ and $\hat{\mathcal{S}}_j$, along with the above marking operations help the framework simulate the open loop transition window.

$$T_{i \rightarrow j} = \hat{\mathcal{S}}_i \cdot h_i + \delta + \hat{\mathcal{S}}_j \cdot h_j \quad (5.25)$$

When the parameters $\hat{\mathcal{S}}_i$, $\hat{\mathcal{S}}_j$, are set to 0 we can simulate the *best-case switch* and when set to $\mathcal{S}_{\text{net}, i}$ and $\mathcal{S}_{\text{net}, j}$ the framework simulates the *worst-case switch*. $T_{i \rightarrow j}^{\text{worst}}$ has an addition h induced from the *hit* packet, whereas $T_{i \rightarrow j}$ only simulates the *miss* patterns.

This helps us validate if the given network reconfiguration delay δ satisfies the Equation (5.6) for a given network configurations.

For example, let us assume a mode switch from $\pi_1 \rightarrow \pi_2$ where π_1 has stream 1 active and π_2 has stream 2 active. Figure 5.5 represents the markings of the places when mode π_1 is active. The stream places of 1 are initialized to generate packets for it. When switched to mode π_2 , the


 Figure 5.6: Mode π_j : stream 2 active, stream 1 inactive.

markings are as shown by Figure 5.6, where the places of stream 2 is initialized and stream 1 made inactive by removing the tokens.

5.4 Reporting

To evaluate the sufficient condition from Equation (5.6) and Equation (4.6), we need to compute the end-to-end latency of each packet traversing through the Net $\mathcal{N}_{tcpn}$. The reporting mechanism observes these events during simulation and constructs the delivery trace μ .

The simulation of the $\mathcal{N}_{tcpn}$ produces a sequence of fired bindings. We extract the temporal properties of the token by defining monitors on the transitions. In our model, we use the data collection monitors as defined in [28]. Monitors are mechanism by which we observe active transitions during simulation and log the details regarding the binding and use it for simulation analysis.

5.4.1 Data Collection Monitor

We define the data collection monitors by introducing the monitor functions required:

Transitions to be monitored are defined as $\mathcal{T}_{\mathcal{M}}$, these are the transitions that the monitor listens on. In this framework, we are interested in transitions responsible for packet generation and those that report the completion of a packet.

$$\mathcal{T}_{\mathcal{M}} = \{t_j^{\text{gen}}, t_j^{\text{done}} \mid f_j \in \text{streams}(i)\} \quad (5.26)$$

Predicate function determines if any of the active bindings in the simulation needs to be observed. For a transition $t \in \mathcal{T}$ at a particular time in the simulation, the monitor observes this transition if and only if $t \in \mathcal{T}_{\mathcal{M}}$.

Observation function determines what needs to be observed from the transition. When the predicate function is satisfied the observation function logs details about the token, which are later used for analysis. In the framework, we model the observation function to track the release times and completion times of packets in their respective transitions.

The monitor is active during the entire duration of the simulation. A packet generated via the transition is observed and release time is logged by the monitor using the mode, stream and packet as index. When the packet reaches the destination, the corresponding transition is active and triggers the monitor to observe the completion time of the packet and log it against the existing index for the packet. Additionally, we can compute the end-to-end latency and other necessary details along with this. This log can then be used to validate the sufficient condition.

5.5 Validation

The data collection monitors from Section 5.4.1 records packet timing information in logs indexed by (π_i, f_l) . This section describes how these observations are used to validate the Equation (4.6) and Equation (5.6).

5.5.1 Single-Mode Validation

For each stream f_l in mode π_i , we compute the end-to-end latency for each packet k as λ_k and evaluate it against the $deadline(l)$.

$$\mu_k = \begin{cases} hit & \text{if } \lambda_k \leq deadline(j) \\ miss & \text{else} \end{cases} \quad (5.27)$$

Equation (5.27) helps us create a sequence of *hit* and *miss*, which is used to validate the sufficient condition for single mode. The μ pattern of packets is collected across all the streams to calculate

the maximum consecutive *miss* patterns $C_{\text{miss},j}^{\text{max}}$, i.e.,

$$C_{\text{miss},j}^{\text{max}} = \max\{\ell \in \mathbb{N} \mid \exists k : \mu_k, \mu_{k+1}, \dots, \mu_{k+\ell-1} = \text{miss}\} \quad (5.28)$$

The sufficient condition for the single mode is satisfied if, for the stream f_j in mode π_i :

$$C_{\text{miss},j}^{\text{max}} \leq \mathcal{S}_{\text{app},i} \quad (5.29)$$

5.5.2 Mode-Switch Validation

For a transition from mode $\pi_i \rightarrow \pi_j$, the validator checks the completion time of packet across both modes to check the transition window.

$$T_{i \rightarrow j}^{\text{obs}} = t_{\text{first-hit}}^{(j)} - t_{\text{last-hit}}^{(i)} \quad (5.30)$$

where $t_{\text{first-hit}}^{(j)}$ represents the time of arrival of the first packet in mode π_j and the $t_{\text{last-hit}}^{(i)}$ represents the time of arrival of the last packet in the mode π_i . The sufficient condition for safety during mode switch is satisfied if:

$$T_{i \rightarrow j}^{\text{obs}} \leq \min\{\mathcal{S}_{\text{app},i} \cdot h_i, \mathcal{S}_{\text{app},j} \cdot h_j\} \quad (5.31)$$

When the framework is configured with $\hat{\mathcal{S}}_i = \mathcal{S}_{\text{net},i}$ and $\hat{\mathcal{S}}_j = \mathcal{S}_{\text{net},j}$, we are able to simulate the worst-case as observed window $T_{i \rightarrow j}^{\text{obs}}$ reaches $T_{i \rightarrow j}^{\text{miss}}$. If the condition holds for the worst-case, we are able to guarantee that the system remains safe for the given network reconfiguration time δ .

5.6 Implementation

The framework is implemented using SimPN [12], a Python-based simulator² that helps us construct a TCPN and simulate it. The SimPN works by calculating the set of transitions enabled at any given instant, then executing each enabled transition based on its timestamp, and triggering the corresponding monitor for observation. The construction proceeds as follows:

1. for each stream f_j in the configuration, the framework instantiates the stream-level places and transitions defined in Section 5.2.2
2. for each edge $(u, v) \in E$, the framework creates the routing transitions.
3. initial marking is set according to the first mode in the mode switch sequence Π

²<https://bpogroup.github.io/simpn/>

4. the set of generation and delivery transitions are registered with the data collector monitor

Each transition is associated with a function that defines how tokens are consumed and produced when the transition fires. These functions realize the arc expressions and timed delays from Section 5.2.3

The algorithm Algorithm 5.1 represents the simulation of the Petri Net and the mode switch functionality. The mode switch algorithm is implemented as an external manipulation of the firing cycle of SimPN. Between t_{app} and t_{net} , the network node transitions still consume π_i from p_v^{mode} and apply schedule τ_i , realizing the transient window described in Section 5.1. The implementation of the framework is open source and available online³.

5.7 Example: Configuration to TCPN model

In this section, we construct a $\mathcal{N}_{\text{tcpn}}$ for a minimal system model configured in JSON. Consider the following JSON configuration as the system under consideration:

```

1  {
2    "nodes": [
3      {"id": 100, "type": "ES"},
4      {"id": 101, "type": "ES"},
5      {"id": 1, "type": "NN"} // Vertices of graph G
6    ],
7    "links": [
8      {"src": 100, "dst": 1},
9      {"src": 1, "dst": 101} // Edges of graph G
10   ],
11   "streams": [
12     {"id": 1, "src": 100, "dst": 101,
13      "period": 0.020, "deadline": 0.020} // describes a stream from node 100 to 101 with a
14     period of 20
15   ],
16   "modes": {
17     "1": {"streams": [1], "schedule_id": 1} // modes of operation
18   },
19   "schedules": {
20     "1": {"pattern": [1, 0], "hit_delay": 0.019} // hit delay of 19ms
21   },
22   "link_delay": [
23     {"prob": 1.0, "delay": 0.001} // assume a constant delay of 1ms
24   ]
25 }
```

Listing 5.1: Minimal system configuration

³<https://github.com/kss682/co-design-validation-framework>

Table 5.2: Mapping of JSON fields to System Model

JSON field	System model
nodes	$V_{ES} = 100, 101, V_{NN} = 1$
links	$E = (100, 1), (1, 101)$
streams	$f_1 = (((100, 1), (1, 101)), h=0.02, d=0.02)$
modes	$\pi_1, streams(1) = 1$
schedules	$sched(1) = (1, 0), dhit = 0.019$
link_delay	$d(u, v) = 0.001 \quad \forall (u, v) \in E : u \in V_{ES}$

The table below maps Listing 5.1 to the system model defined in Chapter 4.

Figure 5.7 shows the TCPN generated using the system configuration in Listing 5.1.

- Places related to V_{ES} : $p_{100}^{es}, p_{101}^{es}$
- Places related to V_{NN} : $p_1^{nn}, p_1^{nn_mode}$
- Places related to stream f_1 : $p_1^{mode}, p_1^{stream}, p_1^{packet}, p_1^{done}$
- Transitions: generate_event_1 for periodic packet generation, t_es_1 for the forwarding of packet from ES, t_nn_1_101 egress of the network node 1 to the ES 101, done_event_1 for observing completed packets from stream 1.

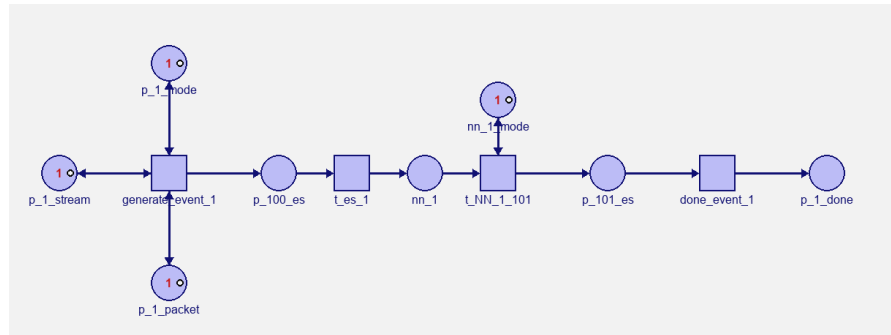


Figure 5.7: CPN constructed from the minimal configuration in Listing 5.1.

Table 5.3: Marking evolution for the first three packets under schedule $\text{sched}(1) = (1, 0)$.

Event	Network places				Stream places			
	p_{100}^{cs}	p_1^{nn}	$p_1^{\text{mode(NN)}}$	p_{101}^{cs}	p_1^{mode}	p_1^{pkt}	p_1^{str}	p_1^{done}
I	$\emptyset$	$\emptyset$	$1@0$	$\emptyset$	$1@0$	$\text{pkt}_1@0$	$1@0$	$\emptyset$
generate_event_1	$\text{pkt}_1@0$	$\emptyset$	$1@0$	$\emptyset$	$1@0.02$	$\text{pkt}_2@0.02$	$1@0.02$	$\emptyset$
t_es_1	$\emptyset$	$\text{pkt}_1@0.001$	$1@0$	$\emptyset$	$1@0.02$	$\text{pkt}_2@0.02$	$1@0.02$	$\emptyset$
t_nn_1_101 (hit)	$\emptyset$	$\emptyset$	$1@0$	$\text{pkt}_1@0.020$	$1@0.02$	$\text{pkt}_2@0.02$	$1@0.02$	$\emptyset$
done_event_1	$\emptyset$	$\emptyset$	$1@0$	$\emptyset$	$1@0.02$	$\text{pkt}_2@0.02$	$1@0.02$	$\text{pkt}_1@0.020$
generate_event_1	$\text{pkt}_2@0.02$	$\emptyset$	$1@0$	$\emptyset$	$1@0.04$	$\text{pkt}_3@0.04$	$1@0.04$	$\text{pkt}_1@0.020$
t_es_1	$\emptyset$	$\text{pkt}_2@0.021$	$1@0$	$\emptyset$	$1@0.04$	$\text{pkt}_3@0.04$	$1@0.04$	$\text{pkt}_1@0.020$
t_nn_1_101 (miss)	$\emptyset$	$\emptyset$	$1@0$	$\emptyset$	$1@0.04$	$\text{pkt}_3@0.04$	$1@0.04$	$\text{pkt}_1@0.020$
generate_event_1	$\text{pkt}_3@0.04$	$\emptyset$	$1@0$	$\emptyset$	$1@0.06$	$\text{pkt}_4@0.06$	$1@0.06$	$\text{pkt}_1@0.020$
t_es_1	$\emptyset$	$\text{pkt}_3@0.041$	$1@0$	$\emptyset$	$1@0.06$	$\text{pkt}_4@0.06$	$1@0.06$	$\text{pkt}_1@0.020$
t_nn_1_101 (hit)	$\emptyset$	$\emptyset$	$1@0$	$\text{pkt}_3@0.060$	$1@0.06$	$\text{pkt}_4@0.06$	$1@0.06$	$\text{pkt}_1@0.020$
done_event_1	$\emptyset$	$\emptyset$	$1@0$	$\emptyset$	$1@0.06$	$\text{pkt}_4@0.06$	$1@0.06$	$\text{pkt}_1@0.020, \text{pkt}_3@0.060$

- Initial Markings:

$$\begin{aligned}
 \mathcal{J}(p_1^{\text{mode}}) &= 1^*1 @ 0 \\
 \mathcal{J}(p_1^{\text{stream}}) &= 1^*1 @ 0 \\
 \mathcal{J}(p_1^{\text{packet}}) &= 1^*\langle 1, 1, 1 \rangle @ 0 \\
 \mathcal{J}(p_1^{\text{nn-mode}}) &= 1^*1 @ 0
 \end{aligned} \tag{5.32}$$

The table Table 5.3 traces the first three packets in the Petri Net. The network node has a schedule $\text{sched}(1) = (1, 0)$, which results a *hit* for the first and third packet and a *miss* for the second packet. The transition *generate_event_1* at $t = 0$, pkt_1 is moved into the node p_{100}^{cs} and generates a packet pkt_2 in p_1^{pkt} with time 0.20. This helps us activate *generate_event_1* in the next period, since pkt_2 becomes active only at time 0.020. At the end of this simulation, pkt_1 and pkt_2 are collected at p_1^{done} at times 0.020 and 0.060.

Algorithmus 5.1 Simulation loop with mode switch

Require: Constructed TCPN, mode switch sequence Π , parameters $\hat{\mathcal{S}}_i, \hat{\mathcal{S}}_j, \delta$ **Require:** $\mathcal{T}_{\mathcal{M}}$

```
1: Compute  $t_{\text{app}}$  and  $t_{\text{net}}$  from  $\Pi, \hat{\mathcal{S}}_i, \delta$ 
2:  $\text{app\_switched} \leftarrow \text{FALSE}$ 
3: while  $t_{\text{clock}} < t_{\text{end}}$  do
4:   if  $\neg \text{app\_switched}$  and  $t_{\text{clock}} \geq t_{\text{app}}$  then
5:      $\text{APPRECONFIGURE}(\pi_j, t_{\text{app}}, \hat{\mathcal{S}}_j)$ 
6:      $\text{app\_switched} \leftarrow \text{TRUE}$ 
7:   if  $\text{app\_switched}$  and  $t_{\text{clock}} \geq t_{\text{net}}$  then
8:      $\text{NETRECONFIGURE}(\pi_j, t_{\text{net}})$ 

9:   Compute enabled transitions
10:  if no enabled transition then
11:    break
12:  Select transition with earliest timestamp
13:  Fire transition, advance  $t_{\text{clock}}$ 
14:  Observe tokens, if transition in  $\mathcal{T}_{\mathcal{M}}$ 

15: function  $\text{APPRECONFIGURE}(\pi_j, t_{\text{app}}, \hat{\mathcal{S}}_j)$ 
16:   for each periodic stream  $f_l$  do
17:      $M(p_l^{\text{mode}}) \leftarrow \emptyset$ 
18:      $M(p_l^{\text{pkt}}) \leftarrow \emptyset$ 
19:      $M(p_l^{\text{str}}) \leftarrow \emptyset$ 
20:   for each  $f_l \in \text{streams}(\pi_j)$  do
21:      $M(p_l^{\text{mode}}) \leftarrow 1' \pi_j @ t_{\text{app}}$ 
22:      $M(p_l^{\text{pkt}}) \leftarrow 1' \langle 1, l, j \rangle @ t_{\text{app}}$ 
23:      $M(p_l^{\text{str}}) \leftarrow 1' l @ t_{\text{app}}$ 

24: function  $\text{NETRECONFIGURE}(\pi_j, t_{\text{net}})$ 
25:   Set cyclic counter of  $\tau_j$  to position  $\hat{\mathcal{S}}_j$ 
26:   for each  $v \in V_{\text{NN}}$  do
27:      $M(p_v^{\text{mode}}) \leftarrow 1' \pi_j @ t_{\text{net}}$ 
```

6 Evaluation

The Co-Design Framework is evaluated using the inverted pendulum on a cart system. We evaluate the mode transition scenarios on the system and validate the safety of the system during the worst-case switching time. We first define the experiment setup in Section 6.1. In Section 6.2, we establish the safety of the system for each mode of operation separately. These results help us establish a sufficient condition defined by the maximum consecutive deadlines missed. In Section 6.3, we evaluate the mode transition window on the best and worst-case scenarios to evaluate our sufficient condition for mode switch. Finally, Section 6.4 further establishes the correlation between the transition point and the relative safety of the system.

6.1 Experimental Setup

In this section, we introduce the dynamics of the inverted pendulum on cart in Section 6.1.1. In Section 6.1.2, we define the behavior, modes, and the stability bound for each mode for the system. Section 6.1.3 establishes the network model, and finally Section 6.1.4 details the experiment process.

6.1.1 Plant: Inverted Pendulum on Cart

We assume a physical pendulum and cart with the specifications as shown in Figure 6.1. The above system is in an open loop, and the pendulum tends to fall without any active control. We balance the pendulum upright by closing the loop with a control force. In this experiment, we aim to bring the cart to a target position while maintaining the pendulum angle $\theta \approx 0$.

To stabilize the system, we linearize it by assuming that the $\theta \approx 0$. Under this assumption, it is essential that the θ remains within a small range so that the linear approximation remains valid. Linearization around $\theta \approx 0$ results in the continuous LTI form as in Equation (2.1) with state

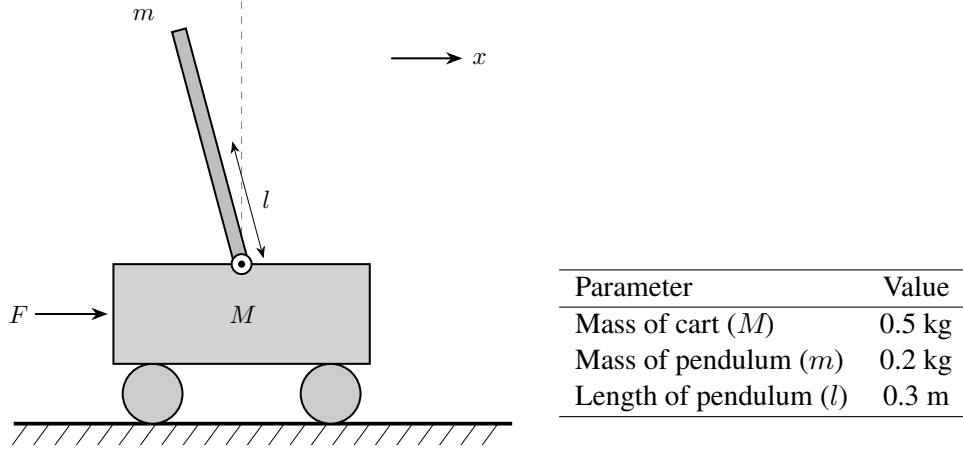


Figure 6.1: Inverted pendulum control system

$x = [p, \dot{p}, \theta, \omega]^4$. We derive the A and B matrices for the system as in [39]:

$$A = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & -\frac{m^2 gl^2}{D} & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & \frac{(M+m)mgl}{D} & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 \\ \frac{J}{D} \\ 0 \\ -\frac{ml}{D} \end{bmatrix} \quad (6.1)$$

where $J = I_{\text{com}} + m \cdot l^2$ and $D = (M + m) \cdot J - m^2 \cdot l^2$. The trajectories of the non-linear inverted pendulum system are close to the trajectories of the linearized model in a small neighborhood around the equilibrium point [30, Ch. 2]. The approximation $\sin \theta \approx \theta$ has a relative error of $\theta^2/6$ which remains below 1.2% for $|\theta| < 15^\circ$. We define this $\theta_{\max} = 15^\circ \approx 0.262$ rad as the safety bound. The physical system is considered safe if $\max |\theta| \leq \theta_{\max}$. The system is designed to have a target state at $x_{\text{target}} = [0, 0, 0, 0]$. When the system is initially in the state x_0 , then the controller calculates the control force in the direction of x_{target} . To calculate the worst-case trajectory for our experiment, we initialize the x_0 in the direction of the dominant eigenvector of the discrete state matrix of A . This eigenvector represents the direction along which the uncontrolled plant diverges the fastest [6].

6.1.2 Modes of Operation

We first define two desired behaviors for the inverted pendulum on cart. The first behavior $\mathcal{B}_1$, the system corrects all four state values with equal and identical weight in the Q matrix and a unit value for R . The second behavior $\mathcal{B}_2$, the system should react aggressively to small deviations

⁴ $p, \dot{p}, \theta, \omega$ represents the distance, velocity, angle and angular velocity respectively

Table 6.1: LQR weight matrices per control mode.

Behavior	Q	R
Baseline	diag(1, 1, 1, 1)	1
Aggressive	diag(100, 10, 100, 10)	0.1

Table 6.2: LQR tuning parameters per control mode.

Mode	Behavior	Gain vector K	h	$\mathcal{S}_{\text{app}}$
π_1	Baseline	$[-0.8665, -1.6880, -18.7274, -3.5930]$	20 ms	7
π_2	Aggressive	$[-23.7709, -21.2328, -88.1440, -18.4465]$	10 ms	3

in position and angle. The design of the Q and R has been selected in Table 6.1 such a way that the system exhibits the desired behavior $\mathcal{B}_1$ and $\mathcal{B}_2$, respectively. For each behavior, we devise appropriate modes of operations π_1, π_2 to express the behavior $\mathcal{B}_1$ and $\mathcal{B}_2$ by defining the sampling period h , gain matrix K , and the tolerance to missing control updates $\mathcal{S}_{\text{app}}$.

The formulation of modes and $\mathcal{S}_{\text{app}}$ is computed by iterating each behavior over a range of sampling period h . For each h , the continuous system matrices are discretized⁵, to obtain A_d and B_d , and the LQR⁶ gain K is computed for the discrete system. The A_{hit} and A_{miss} are constructed as defined in Section 2.1.3. The $\mathcal{S}_{\text{app}}$ is evaluated by iterating from $i = 1$ to find the largest value of i for which the $\max_l |\lambda_l (A_{\text{miss}}^i A_{\text{hit}})| < 1$ as defined in Equation (2.9). This sweep is implemented using the NumPy [20] and Python Control [16] libraries.

The Figure 6.2 shows that the behavior $\mathcal{B}_1$ remains more tolerant to packet loss than $\mathcal{B}_2$. We are able to select appropriate sampling intervals for the modes π_1, π_2 . The system is designed to operate mode π_1 at 20 ms, providing tolerance of at most 7 consecutive deadline misses. The mode π_2 is designed at 10 ms with stability up to 3 misses. Table 6.2 represents the designed modes of operation for the inverted pendulum system. Hence, for the system to remain stable within their respective modes of operations it is necessary that the network is able to guarantee the packet delivery and is able to close the loop before the system becomes unstable. Let τ_1 be the network schedule while the system is operating under π_1 . For τ_1 to guarantee application-level stability, $\tau_1 \vdash \langle \mathcal{S}_{\text{net}, 1} \rangle$, where $\mathcal{S}_{\text{net}, 1} \leq 7$. Similarly, $\tau_2 \vdash \langle \mathcal{S}_{\text{net}, 2} \rangle$ where $\mathcal{S}_{\text{net}, 2} \leq 3$.

6.1.3 Network Model

⁵<https://python-control.readthedocs.io/en/latest/generated/control.matlab.c2d.html>

⁶<https://python-control.readthedocs.io/en/latest/generated/control.dlqr.html>

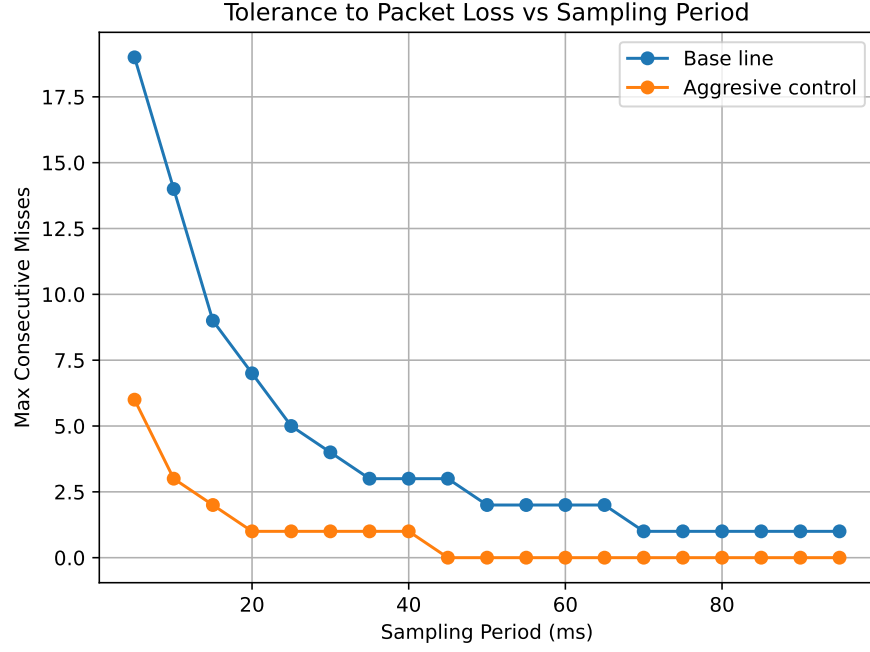


Figure 6.2: Maximum Consecutive Misses vs Sampling Period

The system model is defined for the inverted pendulum using the semantics from Chapter 4. The topology consists of three nodes connected in a linear topology. There are two ES nodes representing the sensor node and the controller and actuator in the other node. The ES are connected through a NN. The sensor node $ES(id = 100)$ samples the current state of the system to the controller in $ES(id = 101)$. The network node $NN(id = 1)$ guarantees that the deliverance of the packets through the network schedule.

For each mode π_i , there is a dedicated time-triggered stream f_i from $ES(100)$ to $ES(101)$ with a period of h_i . There exists a valid network schedule τ_i for each mode that guarantees the upper bound on the consecutive deadline miss due to the network node. τ_i is represented by $\mathcal{S}_{net,i}$ where $\mathcal{S}_{net,i} \leq \mathcal{S}_{app,i}$. Listing 6.1 represents a configuration for the system in which the network schedule τ_1 for mode π_1 guarantees that the maximum consecutive deadline miss will be $\mathcal{S}_{net,1} = 2 \leq \mathcal{S}_{app,1} = 7$. The listing shows the system starting in π_2 until $t = 40$ ms when the system switches to π_1 . The reconfiguration delay δ is provided as an external parameter to the Petri Net simulator.

```

1 {
2   "modes": {
3     "1": {"name": "baseline", "streams": [1], "schedule_id": 1},
4     "2": {"name": "aggressive", "streams": [3], "schedule_id": 2}
5   },
6   "schedules": {
7     "1": {"pattern": [1, 0, 0], "hit_delay": 0.018},
8     "2": {"pattern": [1], "hit_delay": 0.008}
9   },
10  "mode_switch": [
11    [2, 0.000],
12    [1, 0.040]
13  ],
14  "link_delay": [
15    {"prob": 0.99, "delay": 0.001},
16    {"prob": 0.01, "delay": 0.002}
17  ]
18 }

```

Listing 6.1: Configuration for the two mode pendulum cart system

6.1.4 Experiment Process

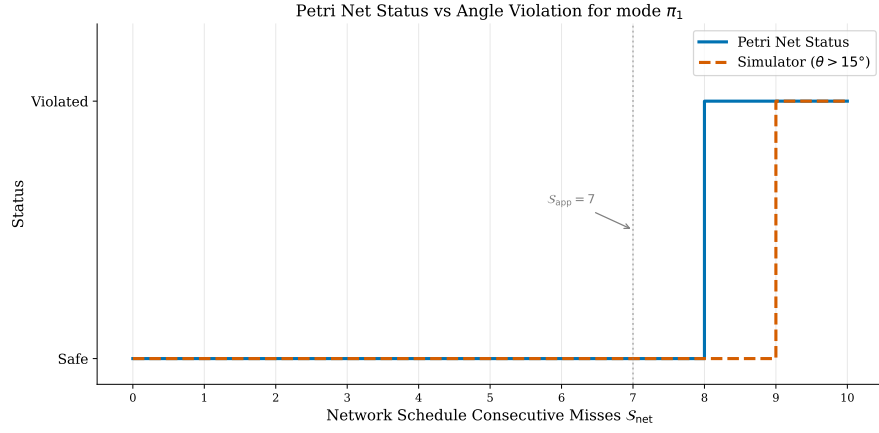
The experiment is conducted by passing a JSON similar to Listing 6.1 to the Petri Net validator from Chapter 5. The simulator models the flow of packets from $ES(100)$ to $ES(101)$ through the network topology. The network schedules of the modes are enforced at $NN(1)$. The reporters from the simulator observe the packets on the transition node in the Petri Net that are being monitored. The validator checks the observed trace against the upper bound for safety for individual mode of operation and mode transition window, as defined in Equation (4.6) and Equation (5.6). If the simulator observes unsafe state for duration greater than the upper bound, the reporter indicates the system has a run that leads to an unsafe state.

The Petri Net simulator monitor traces of packets received at the ES nodes. These traces are used by the event-based system simulator to record the system's state. The inverted pendulum simulator integrates the non-linear system equations under ZOH using the explicit Runge-Kutta method of order 4 [13] via the *solve_ivp*⁷ method in the SciPy Python library [46]. The output is a state trajectory of the system from which the $\max |\theta|$ is extracted.

6.2 Single Mode Stability

Before analyzing the stability of the system during mode switch, we first evaluate the sufficient condition for single mode operation to establish the base condition. For each mode π_i designed using the LQR gain matrix K_i and sampling period h_i , we have derived an upper bound on the

⁷https://docs.scipy.org/doc/scipy/reference/generated/scipy.integrate.solve_ivp.html

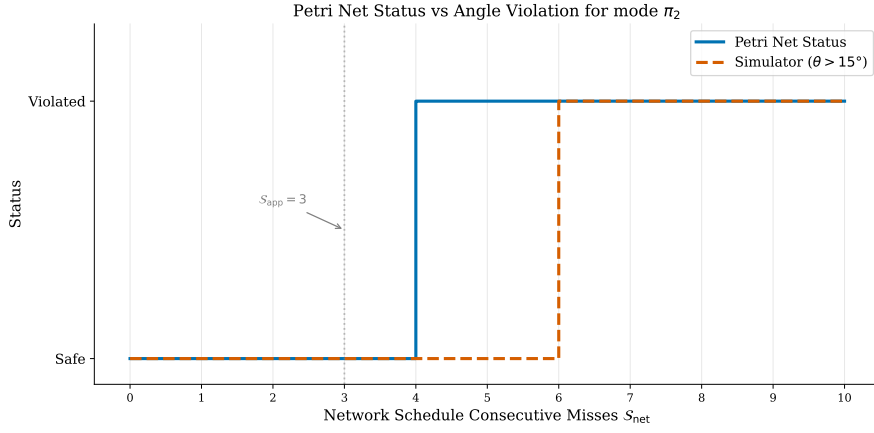
Figure 6.3: Stability analysis for π_1

maximum consecutive deadline misses for that mode as $\mathcal{S}_{\text{app}, i}$. In this experiment, we prove that $\mathcal{S}_{\text{net}, i} \leq \mathcal{S}_{\text{app}, i}$ (Equation (4.6)) is a sufficient condition for safety of the system.

In the experiment, the modes π_1 and π_2 with $\mathcal{S}_{\text{app}, 1} = 7$ and $\mathcal{S}_{\text{app}, 2} = 3$ are analyzed. We analyze the results by simulating the Petri Net under a network schedule τ with $\mathcal{S}_{\text{net}, i}$ iterated from $0 \rightarrow 10$, this helps us simulate different worst-case deadline miss patterns for the modes. The Petri Net simulator is run for 20 s and reports if the sufficient condition is *Satisfied* or *Violated*. The resulting packet trace is used to run the pendulum simulator whose results are then used to check the angle deviation for system safety. We aim to validate that as long as the Petri Net simulator reports *Satisfied* the corresponding pendulum simulator always remains safe.

Mode π_1 Figure 6.3 represents the analysis for mode π_1 , for all $\mathcal{S}_{\text{net}, 1} \leq 7$ the Petri Net mode reports *Safe* as well as the angle of the pendulum simulator remains within $\theta_{\text{max}} = 0.262$ rad. During this simulation, the system is able to compensate the open loop evolution due to the miss packets. But from $\mathcal{S}_{\text{net}, 1} = 8$ the Petri Net simulator is unable to guarantee the safety of the system and from $\mathcal{S}_{\text{net}, 1} = 9$ the angle deviation is large enough to break the linearized model.

Mode π_2 Figure 6.4 represents the analysis for mode π_2 . The result is similar, but the Petri Net simulator reports *Safe* until $\mathcal{S}_{\text{net}, 2} \leq 3$ and the pendulum simulator validates this report as the angle deviation does not exceed $\theta_{\text{max}} = 0.262$ rad. In mode π_2 , due to the aggressive nature of the gain matrix K_2 control force is more intense due to this the safety of the system is guaranteed only until $\mathcal{S}_{\text{app}, 2}$.

Figure 6.4: Stability analysis for π_2

These results validate that the $\mathcal{S}_{\text{net},i} \leq \mathcal{S}_{\text{app},i}$ for each mode is a sufficient condition for ensuring the safety of the system. For any mode π_i operating with network schedule $\tau_i \vdash \langle \mathcal{S}_{\text{net},i} \rangle$ with $\mathcal{S}_{\text{net},i} \leq \mathcal{S}_{\text{app},i}$, the system is safe $\sigma(\pi_i)$.

6.3 Mode Switch Evaluation

In the previous section, we evaluated the safe criteria during a mode of operation. In this section, we evaluate the sufficient condition Equation (5.6) during the transition window. We evaluate this bound by iterating δ over $10 \rightarrow 250$ ms with an increment of 5ms for the transition $\pi_1 \rightarrow \pi_2$ and $\pi_2 \rightarrow \pi_1$, simulating both the best-case $\mathcal{S}_{\text{net},i} = 0$ and worst-case switch time $\mathcal{S}_{\text{net},i} = \mathcal{S}_{\text{app},i}$. The mode switch is triggered when the initial mode has received two control updates.

Best-Case Switch: Figure 6.5 represents the best-case switch when the mode switch occurs immediately after a control packet has reached and the first packet from new mode reaches immediately. The blue curve represents the switch $\pi_1 \rightarrow \pi_2$, it can be observed that the $\max|\theta|$ is within the neighborhood of linearization until the reconfiguration delay reaches ≈ 190 ms, from there the maximum deviation during the miss starts to increase and moves outside the θ_{max} . The red curve represents the switch from $\pi_2 \rightarrow \pi_1$ and is more sensitive to control packet miss. The $\max|\theta|$ starts growing for $\delta \approx 40\text{ms}$. The θ_{max} is held until 60ms and then the system continues to diverge. In both case, the system is safe within the 30ms bound.

Intermediate-Case: Figure 6.6a represents a scenario where the network induces $\mathcal{S}_{\text{net},1} = 3$ and $\mathcal{S}_{\text{net},2} = 1$ consecutive misses. The sufficient condition requires $T_{1 \rightarrow 2}^{\text{obs}} = 3 \cdot 20 + \delta + (1) \cdot 10 = 70 + \delta \leq 30$ ms, which is not satisfiable for any $\delta \geq 0$. However, $\pi_1 \rightarrow \pi_2$ direction remains stable

6 Evaluation

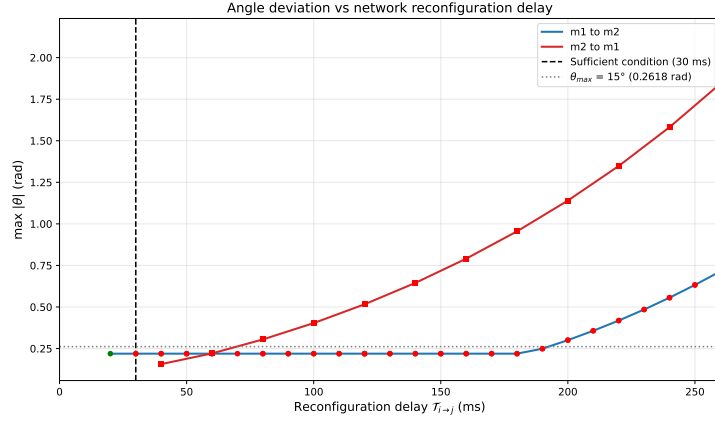
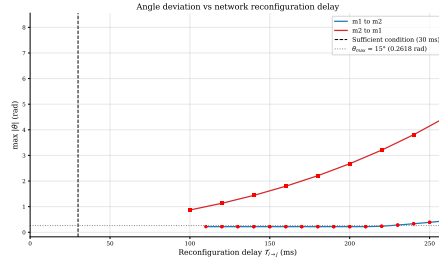
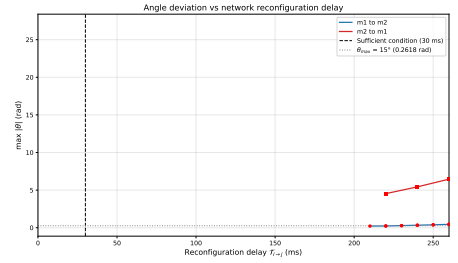


Figure 6.5: Best-Case: mode switch when network guarantees $\mathcal{S}_{\text{net}, i} = 0$



(a) Intermediate-Case: network schedule $\mathcal{S}_{\text{net}, 1} = 3$ and $\mathcal{S}_{\text{net}, 2} = 1$



(b) Worst-Case: network schedule $\mathcal{S}_{\text{net}, 1} = 7$ and $\mathcal{S}_{\text{net}, 2} = 3$

Figure 6.6: Mode switch analysis under different network schedule

until $\delta \approx 210$ ms, while $\pi_2 \rightarrow \pi_1$ is unstable for all tested δ values, as the minimum transition window of 80 ms already exceeds the best-case stability cross-over point of 60 ms

Worst-Case Switch: Figure 6.6b represents the worst-case switch. The minimum time in open loop $\mathcal{T}_{i \rightarrow j}^{\text{obs}} = (7) * 20 + \delta + (3) * 10 = 170 + \delta$. The sufficient condition requires $\mathcal{T}_{i \rightarrow j}^{\text{worst}} \leq 30$ ms, which is not possible for any $\delta > 0$. The system safety can no longer be guaranteed for any $\delta > 0$.

From both analysis, we can understand the switch the $\pi_2 \rightarrow \pi_1$ direction consistently diverges earlier than $\pi_1 \rightarrow \pi_2$. The switch from $\pi_1 \rightarrow \pi_2$ remains bound until $\mathcal{T}_{i \rightarrow j} \approx 190 - 210$ ms before diverging. The switch from $\pi_2 \rightarrow \pi_1$ starts diverging around 55 ms. This asymmetry arises because when switching from π_2 the open loop evolution under the constant control force from K_2 causes larger deviations to delay. For $\pi_1 \rightarrow \pi_2$ and $\pi_2 \rightarrow \pi_1$, the system remains safe until approximately 180 ms to 210 ms and 55 ms to 60 ms respectively, validating the soundness

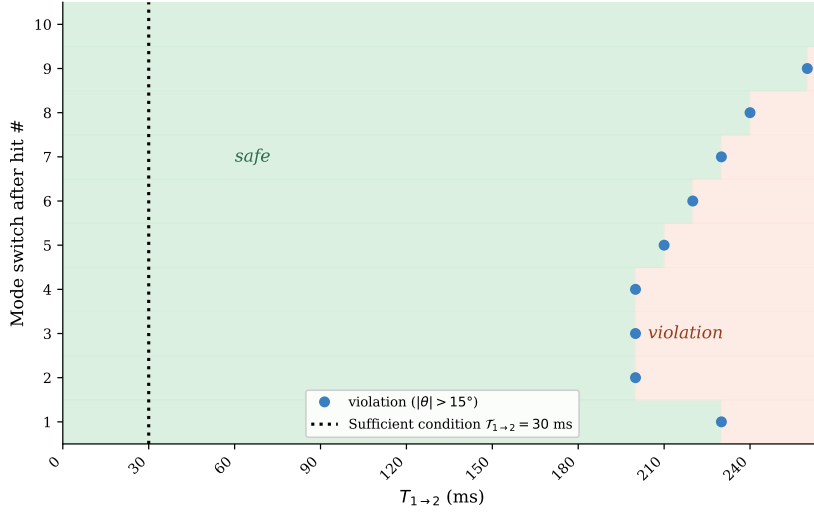


Figure 6.7: Effect of switch time on safety of system during mode switch from $\pi_1 \rightarrow \pi_2$

and conservative nature of the sufficient bound. This would mean direction of switch plays a role in deciding the conservativeness.

The analysis shows that the $T_{i \rightarrow j}^{\text{obs}} < 30$ ms only for the best-case switch for the $\pi_1 \rightarrow \pi_2$. To incorporate safety during all mode transition, the network parameters of $\mathcal{S}_{\text{net}}$ and reconfiguration delay δ needs to be tuned to ensure the $T_{i \rightarrow j}^{\text{worst}} < 30$ ms for both directions. If no feasible schedule satisfying this bound exists, it calls for the redesign of the modes of operation. The framework enables this iterative redesign by allowing the designer to evaluate each configuration.

6.4 Switch Time Analysis

In this section, we evaluate the effect of the time of mode switch. When the system switches immediately after the first hit, the state might be largely uncorrected along one or more dimensions of the system state, resulting in a much faster divergence from the linearized neighborhood. When the state has received sufficient amount of control updates, it is closer to equilibrium and can tolerate packet over a larger period of time. We denote the switch time by hit index, hit n means the switch is initiated after n successful control update has reached actuator. We iterate over the reconfiguration delay δ from $10 \rightarrow 250$ ms over the hit index from $1 \rightarrow 10$ and observe the $\max|\theta|$ over the transition window.

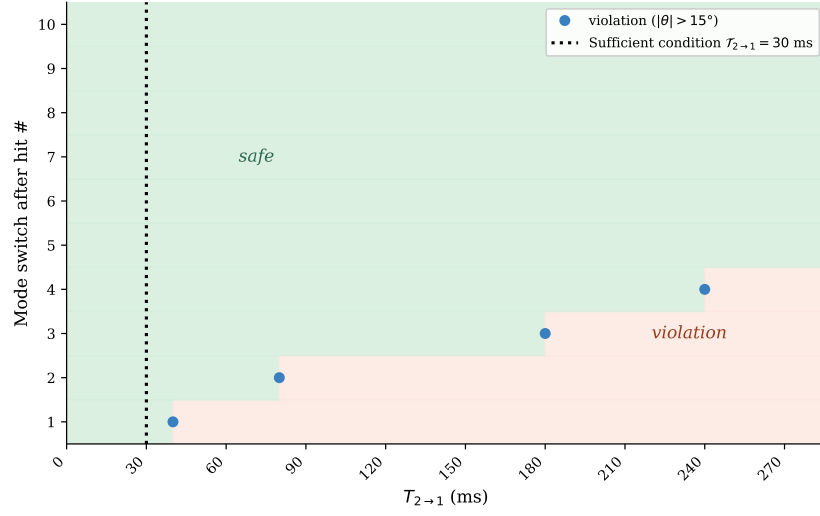


Figure 6.8: Effect of switch on safety of system during mode switch from $\pi_2 \rightarrow \pi_1$

$\pi_1 \rightarrow \pi_2$: In Figure 6.7, the switch time has minimal effect on the stability. All the ten hit index remain stable until $180ms$ before diverging. The control update produced by K_1 is small enough during any point in time that the stability is large enough.

$\pi_2 \rightarrow \pi_1$: In Figure 6.8, the switch has much profound effect on the stability. For switch after the first control update is received the $\theta_{\max}$ is crossed at $40ms$ and after the second update the $\theta_{\max}$ is crossed at $75ms$. From the third control update, the stability is held for longer time and state diverges much more gradually.

This contrast in stability at the switch time is due to the aggressive control force from K_2 . Since π_2 converges with fewer control updates, the first few hits would show large deviations in θ when compared with π_1 . This analysis shows that effective stability during mode switch also depends on the number of control updates received before the mode switch is triggered. These results validate the dwell time concept from switched systems theory [35], deferring the switch until the system state has converged improves robustness to the transition window. The framework enables the user to identify the minimum number of control updates required.

The summary of these experiments can be highlighted as follows:

- **Soundness:** The sufficient conditions hold across all tested configurations, the system state is not violated for bound below 30 ms.
- **Conservativeness:** The system stability violation occurs at a factor of ≈ 2 and ≈ 6 times the safety bound.

- **Asymmetry:** The transition direction influences the extent to which the system is safe beyond the bound.
- **Deferred Switching:** Deferring the mode switch until the system state is stable improves the robustness to transition window.

The framework helps to explore these factors, by choosing the network schedule, reconfiguration delay, mode design to find a suitable configuration that satisfies the sufficient condition for all the required mode transitions.

7 Conclusion

Mode transitions in NCS introduce a safety problem during the transition window. When the application switches mode prematurely, the network reconfigures after a time δ . During this transition window, the current network schedule cannot guarantee the safety of the system in new mode. This thesis answers under what condition of reconfiguration time δ and the network guarantee $\mathcal{S}_{\text{net}}$ the LTI remains safe across a mode transition.

The outline of the work is as follows:

1. We derive a sufficient condition Equation (5.6), that maps the network guarantees $\mathcal{S}_{\text{net}}$, the reconfiguration delay δ , and the control side stability bound $\mathcal{S}_{\text{app}}$. The condition is independent of the transition direction and the system state at the moment of switching.
2. We develop a TCPN validation framework in Chapter 5 that models the packet generation, network schedule and transmission of packets across the network. The delivery trace of the packets is reported, and validated against the sufficient conditions.
3. We evaluate the framework on an inverted pendulum control system for three sets of experiments. One, analysis of the stability of the system for individual modes of operation. Second, studies the mode switch, both $\pi_1 \rightarrow \pi_2$ and $\pi_2 \rightarrow \pi_1$, against the reconfiguration delay. Finally, the analysis of the effect of switch time on the stability.

The sufficient condition was confirmed to be sound for all the scenarios evaluated for the inverted pendulum control system. The experiments observed that the safety of the system is not violated for transition windows below the sufficient bound, and Equation (5.6) is a conservative bound. The transition from aggressive control of π_2 to the baseline control of π_1 , the system safety is observed to violate earlier than the reverse transition $\pi_1 \rightarrow \pi_2$. The aggressive gain of π_2 corrects deviations in system state strongly. During the transition window, when the packets are not guaranteed, the aggressive hold of π_2 is held for the duration of the window, and results in rapid divergence from the linearization.

The switch time analysis show that for the transition from an aggressive mode, switching early causes instability even for a small transition window, while switching late allows the system to reach a state that is close to equilibrium, keeping the system safe for a wide range of transition window. These results align with the dwell-time condition for switched systems in [35], which states that a sufficiently large dwell time guarantees asymptotic stability.

Future Work

The current work limits the study to a single control loop, whereas in practice, multiple control loops share the same network infrastructure. Al-Areqi et al. [4] studies multiple control loops in the absence of network delay. Extending our framework to this setting and considering the network delay, introduces two challenges that we have identified. First, during mode switch on one loop, the delivery guarantees of non-switching loops are disrupted. The sufficient condition must be extended to include all the control loops. Second, simultaneous mode changes across multiple control loops would require a conflict resolution strategy to order the mode change. The TCPN framework can be extended to this setting by modeling multiple loops as concurrent packet token flows.

Another extension could be in the direction of hierarchical control architectures, where a high-level controller determines when to switch modes. In such systems, the switching decision depends on factors such as plant state, performance degradation or external triggers. One open question is whether, by allowing co-simulation of the plant and the TCPN, can we use the behavior properties of the Petri Net such as liveness of a place, to trigger mode switch implicitly within the Net.

Bibliography

- [1] L. Ahrendts, S. Quinton, T. Boroske, R. Ernst. “Verifying Weakly-Hard Real-Time Properties of Traffic Streams in Switched Networks”. In: *LIPICs, Volume 106, ECRTS 2018* 106 (2018). Ed. by S. Altmeyer, 15:1–15:22. ISSN: 1868-8969. DOI: 10.4230/LIPICs.ECRTS.2018.15 (cit. on p. 26).
- [2] M. Ajmone Marsan, G. Conte, G. Balbo. “A class of generalized stochastic Petri nets for the performance evaluation of multiprocessor systems”. In: *ACM Trans. Comput. Syst.* 2.2 (May 1984), pp. 93–122. ISSN: 0734-2071. DOI: 10.1145/190.191. URL: <https://doi.org/10.1145/190.191> (cit. on p. 22).
- [3] R. Alur, D. L. Dill. “A Theory of Timed Automata”. In: *Theoretical Computer Science* 126.2 (Apr. 1994), pp. 183–235. ISSN: 03043975. DOI: 10.1016/0304-3975(94)90010-8 (cit. on p. 26).
- [4] S. Al-Areqi, D. Görge, S. Liu. “Event-Based Networked Control and Scheduling Code-sign with Guaranteed Performance”. In: *Automatica* 57 (July 2015), pp. 128–134. ISSN: 00051098. DOI: 10.1016/j.automatica.2015.04.003 (cit. on p. 66).
- [5] K. J. Åström, T. Hägglund. *PID Controllers: Theory, Design, and Tuning*. 2. ed. Research Triangle Park, NC: Instrument Society of America, 1995. ISBN: 978-1-55617-516-9 (cit. on p. 18).
- [6] K. J. Åström, B. Wittenmark. *Computer-Controlled Systems: Theory and Design*. 3rd. Prentice Hall, 1997. ISBN: 978-0133148992 (cit. on pp. 15, 18, 20, 54).
- [7] D. Baumann, F. Mager, R. Jacob, L. Thiele, M. Zimmerling, S. Trimpe. “Fast Feedback Control over Multi-hop Wireless Networks with Mode Changes and Stability Guarantees”. In: *ACM Transactions on Cyber-Physical Systems* 4.2 (Apr. 2020), pp. 1–32. ISSN: 2378-962X, 2378-9638. DOI: 10.1145/3361846 (cit. on pp. 26, 33).
- [8] G. Bernat, A. Burns, A. Liamosi. “Weakly Hard Real-Time Systems”. In: *IEEE Transactions on Computers* 50.4 (Apr. 2001), pp. 308–321. ISSN: 1557-9956. DOI: 10.1109/12.919277 (cit. on pp. 20, 25).
- [9] R. Blind, F. Allgöwer. “Towards Networked Control Systems with Guaranteed Stability: Using Weakly Hard Real-Time Constraints to Model the Loss Process”. In: *2015 54th IEEE Conference on Decision and Control (CDC)*. Dec. 2015, pp. 7510–7515. DOI: 10.1109/CDC.2015.7403405 (cit. on p. 25).

- [10] A. Cervin, D. Henriksson, B. Lincoln, K.-E. Årzén. “Jitterbug and TrueTime: Analysis Tools for Real-Time Control Systems”. English. In: 2nd Workshop on Real-Time Tools ; Conference date: 01-08-2002 Through 01-08-2002. 2002 (cit. on p. 26).
- [11] T. Chen, L. T. X. Phan. “SafeMC: A System for the Design and Evaluation of Mode-Change Protocols”. In: *2018 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. Apr. 2018, pp. 105–116. DOI: 10.1109/RTAS.2018.00021 (cit. on p. 15).
- [12] R. M. Dijkman. “SimPN: A Python Library for Modeling and Simulating Timed, Colored Petri Nets”. In: () (cit. on p. 48).
- [13] J. R. Dormand, P. J. Prince. “A Family of Embedded Runge-Kutta Formulae”. In: *Journal of Computational and Applied Mathematics* 6.1 (Mar. 1980), pp. 19–26. ISSN: 0377-0427. DOI: 10.1016/0771-050X(80)90013-3 (cit. on p. 57).
- [14] S. Egger, J. Gross, J. Sachs, G. P. Sharma, C. Becker, F. Dürr. *End-to-End Reliability in Wireless IEEE 802.1Qbv Time-Sensitive Networks*. Feb. 2025. DOI: 10.48550/arXiv.2502.11595. arXiv: 2502.11595 [cs] (cit. on pp. 25, 31).
- [15] S. Egger, R. Laidig, H. Geppert, L. Haug, J. Herrmann, F. Dürr, C. Becker. *An (m,k)-Firm Elevation Policy to Increase the Robustness of Time-Driven Schedules in 5G Time-Sensitive Networks*. Aug. 2025. DOI: 10.48550/arXiv.2508.09769. arXiv: 2508.09769 [cs.NI] (cit. on pp. 21, 26).
- [16] S. Fuller, B. Greiner, J. Moore, R. Murray, R. van Paassen, R. Yorke. “The Python Control Systems Library (python-control)”. In: *60th IEEE Conference on Decision and Control (CDC)*. IEEE. 2021, pp. 4875–4881 (cit. on p. 55).
- [17] R. A. Gupta, M.-Y. Chow. “Networked Control System: Overview and Research Trends”. In: *IEEE Transactions on Industrial Electronics* 57.7 (July 2010), pp. 2527–2535. ISSN: 1557-9948. DOI: 10.1109/TIE.2009.2035462 (cit. on p. 17).
- [18] Y. Halevi, A. Ray. “Integrated Communication and Control Systems: Part I—Analysis”. In: *Journal of Dynamic Systems, Measurement, and Control* 110.4 (Dec. 1988), pp. 367–373. ISSN: 0022-0434, 1528-9028. DOI: 10.1115/1.3152698 (cit. on pp. 17, 20).
- [19] M. Hamdaoui, P. Ramanathan. “Hamdaoui, M.: A Dynamic Priority Assignment Technique for Streams with (m,k)-Firm Deadlines. *IEEE Trans. Comput.* 44, 1443-1451”. In: *Computers, IEEE Transactions on* 44 (Jan. 1996), pp. 1443–1451. DOI: 10.1109/12.477249 (cit. on pp. 20, 25).
- [20] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, T. E. Oliphant. “Array programming with NumPy”. In: *Nature* 585.7825 (Sept. 2020), pp. 357–362. DOI: 10.1038/s41586-020-2649-2. URL: <https://doi.org/10.1038/s41586-020-2649-2> (cit. on p. 55).

- [21] K. Hee van, L. Somers, M. Voorhoeve. “Executable specifications for distributed information systems”. English. In: *Information System Concepts (Proceedings of the IFIP TC 8-WG 8.1 Conference, Namur Belgium, October 18-20, 1989)*. Ed. by E. Falkenberg, P. Lindgreen. Netherlands: North-Holland Publishing Company, 1989, pp. 139–156. ISBN: 0-444-88323-1 (cit. on p. 22).
- [22] J. Hespanha, A. Morse. “Stability of Switched Systems with Average Dwell-Time”. In: *Proceedings of the 38th IEEE Conference on Decision and Control (Cat. No.99CH36304)*. Vol. 3. Phoenix, AZ, USA: IEEE, 1999, pp. 2655–2660. ISBN: 978-0-7803-5250-6. DOI: 10.1109/CDC.1999.831330 (cit. on p. 25).
- [23] J. P. Hespanha, P. Naghshtabrizi, Y. Xu. “A Survey of Recent Results in Networked Control Systems”. In: *Proceedings of the IEEE* 95.1 (Jan. 2007), pp. 138–162. ISSN: 0018-9219. DOI: 10.1109/JPROC.2006.887288 (cit. on pp. 15, 18, 19, 25, 29).
- [24] “IEEE Standard for Local and Metropolitan Area Networks – Bridges and Bridged Networks - Amendment 25: Enhancements for Scheduled Traffic”. In: *IEEE Std 802.1Qbv-2015 (Amendment to IEEE Std 802.1Q-2014 as amended by IEEE Std 802.1Qca-2015, IEEE Std 802.1Qcd-2015, and IEEE Std 802.1Q-2014/Cor 1-2015)* (Mar. 2016), pp. 1–57. DOI: 10.1109/IEEESTD.2016.8613095 (cit. on p. 25).
- [25] K. Jensen. “Coloured Petri nets”; in: “Advanced Course on Petri Nets”. In: 254 (1986). Ed. by G. Rozenberg, pp. 248–299 (cit. on p. 21).
- [26] K. Jensen, L. M. Kristensen. “Formal Definition of Timed Coloured Petri Nets”. In: *Coloured Petri Nets: Modelling and Validation of Concurrent Systems*. Ed. by K. Jensen, L. M. Kristensen. Berlin, Heidelberg: Springer, 2009, pp. 257–271. ISBN: 978-3-642-00284-7. DOI: 10.1007/b95112_11 (cit. on pp. 16, 22, 26).
- [27] K. Jensen, L. M. Kristensen. “Introduction to Modelling and Validation”. In: *Coloured Petri Nets: Modelling and Validation of Concurrent Systems*. Ed. by K. Jensen, L. M. Kristensen. Berlin, Heidelberg: Springer, 2009, pp. 1–12. ISBN: 978-3-642-00284-7. DOI: 10.1007/b95112_1 (cit. on p. 37).
- [28] K. Jensen, L. M. Kristensen. “Simulation-Based Performance Analysis”. In: *Coloured Petri Nets: Modelling and Validation of Concurrent Systems*. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 273–301. ISBN: 978-3-642-00284-7. DOI: 10.1007/b95112_12 (cit. on pp. 26, 46).
- [29] T. Kailath. “Linear Systems”. In: Jan. 1980, pp. 230–234 (cit. on p. 19).
- [30] H. K. Khalil. *Nonlinear Systems*. 3rd ed. Upper Saddle River, NJ: Prentice Hall, 2002 (cit. on pp. 32, 54).
- [31] K.-D. Kim, P. R. Kumar. “Cyber–Physical Systems: A Perspective at the Centennial”. In: *Proceedings of the IEEE* 100.Special Centennial Issue (May 2012), pp. 1287–1308. ISSN: 1558-2256. DOI: 10.1109/JPROC.2012.2189792 (cit. on p. 15).

- [32] H. Kwakernaak. *Linear Optimal Control Systems*. New York, NY, USA: John Wiley & Sons, Inc, 1972 (cit. on p. 18).
- [33] W. S. Levine. *The Control Handbook. Electrical Engineering Handbook*. Taylor & Francis, 1996 (cit. on p. 18).
- [34] D. Liberzon. “Switched Systems: Stability Analysis and Control Synthesis”. In: () (cit. on p. 25).
- [35] D. Liberzon, A. S. Morse. “Basic problems in stability and design of switched systems”. In: *IEEE Control Systems Magazine* 19 (1999), pp. 59–70. URL: <https://api.semanticscholar.org/CorpusID:12706115> (cit. on pp. 25, 62, 65).
- [36] J. W. S. W. Liu. *Real-Time Systems*. 1st. USA: Prentice Hall PTR, 2000. ISBN: 0130996513 (cit. on p. 21).
- [37] M. Maggio, A. Hamann, E. Mayer-John, D. Ziegenbein. “Control-System Stability Under Consecutive Deadline Misses Constraints”. In: *LIPICs, Volume 165, ECRTS 2020* 165 (2020). Ed. by M. Völz, 21:1–21:24. ISSN: 1868-8969. DOI: 10.4230/LIPICs.ECRTS.2020.21 (cit. on pp. 15, 20, 25).
- [38] J. Nilsson. “Real-Time Control Systems with Delays”. PhD thesis. Department of Automatic Control, Lund Institute of Technology (LTH), 1998 (cit. on p. 19).
- [39] K. Ogata. *Modern Control Engineering*. 5th. Prentice Hall, 2010 (cit. on pp. 17, 54).
- [40] C. A. Petri. *Kommunikation mit Automaten*. Dissertation, Schriften des IIM 2. Bonn: Rheinisch-Westfälisches Institut für Instrumentelle Mathematik an der Universität Bonn, 1962 (cit. on pp. 21, 26).
- [41] “Proc. of Workshops on Practical Use of Coloured Petri Nets and the CPN Tools.” In: (1998). URL: <https://cs.au.dk/cpnets/> (cit. on p. 26).
- [42] L. Schenato, B. Sinopoli, M. Franceschetti, K. Poolla, S. S. Sastry. “Foundations of Control and Estimation Over Lossy Networks”. In: *Proceedings of the IEEE* 95.1 (Jan. 2007), pp. 163–187. ISSN: 0018-9219. DOI: 10.1109/JPROC.2006.887306 (cit. on pp. 15, 25).
- [43] P. Seiler, R. Sengupta. “Analysis of Communication Losses in Vehicle Control Problems”. In: *Proceedings of the 2001 American Control Conference. (Cat. No.01CH37148)*. Vol. 2. June 2001, 1491–1496 vol.2. DOI: 10.1109/ACC.2001.945935 (cit. on p. 15).
- [44] F. Tamssaouet, S. Amari. “Modelling and temporal evaluation of networked control systems using timed automata with guards and (max,+) algebra”. In: *International Journal of Systems Science* 49.10 (2018), pp. 2073–2088. DOI: 10.1080/00207721.2018.1483541. URL: <https://doi.org/10.1080/00207721.2018.1483541> (cit. on p. 26).
- [45] W. M. P. Van Der Aalst. *Timed coloured Petri nets and their application to logistics*. Eindhoven, 1992 (cit. on p. 21).

- [46] P. Virtanen, R. Gommers, T. E. Oliphant, M. Haberland, T. Reddy, D. Cournapeau, E. Burovski, P. Peterson, W. Weckesser, J. Bright, S. J. van der Walt, M. Brett, J. Wilson, K. J. Millman, N. Mayorov, A. R. J. Nelson, E. Jones, R. Kern, E. Larson, C. J. Carey, Í. Polat, Y. Feng, E. W. Moore, J. VanderPlas, D. Laxalde, J. Perktold, R. Cimrman, I. Henriksen, E. A. Quintero, C. R. Harris, A. M. Archibald, A. H. Ribeiro, F. Pedregosa, P. van Mulbregt, SciPy 1.0 Contributors. “SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python”. In: *Nature Methods* 17 (2020), pp. 261–272. DOI: 10.1038/s41592-019-0686-2 (cit. on p. 57).
- [47] N. Vreman, A. Cervin, M. Maggio. “Stability and Performance Analysis of Control Systems Subject to Bursts of Deadline Misses”. In: *LIPICs, Volume 196, ECRTS 2021* 196 (2021). Ed. by B. B. Brandenburg, 15:1–15:23. ISSN: 1868-8969. DOI: 10.4230/LIPICS.ECRTS.2021.15 (cit. on p. 20).
- [48] C. R. Zervos. *Coloured Petri Nets: their properties and applications*. Michigan, 1977 (cit. on p. 21).
- [49] W. Zhang, M. Branicky, S. Phillips. “Stability of networked control systems”. In: *IEEE Control Systems Magazine* 21.1 (2001), pp. 84–99. DOI: 10.1109/37.898794 (cit. on pp. 15, 19, 25, 29).
- [50] W. M. Zuberek. “Timed Petri Nets Definitions, Properties, and Applications”. In: *Microelectronics Reliability* 31.4 (Jan. 1991), pp. 627–644. ISSN: 0026-2714. DOI: 10.1016/0026-2714(91)90007-T (cit. on p. 21).
- [51] W. Zuberek. “Throughput Analysis in Timed Colored Petri Nets”. In: *1993 IEEE International Symposium on Circuits and Systems*. Chicago, IL, USA: IEEE, 1993, pp. 2721–2724. ISBN: 978-0-7803-1281-4. DOI: 10.1109/ISCAS.1993.394329 (cit. on p. 21).

A Use of AI Tools

This appendix documents the use of AI tools during the preparation of this thesis, in accordance with the principles for the use of AI by students at the University of Stuttgart. During the course of thesis, Claude AI⁸ was used in various phases.

During the literature survey, Claude was used to assist in searching relevant academic papers along with other sources like Google. All sources were independently retrieved and verified from their original publications.

During the implementation of framework, Claude usage was limited to debugging issues. AI has not been used to generate framework code. For the evaluations, Claude was used to assist in generating the visualization and debug the shell script for the benchmarking. AI data has not been used to plot.

Throughout the thesis writing process, Claude was used to proofread content for grammatical, spelling, and stylistic issues. Claude help was sought while drawing the tikz diagrams. No AI assistance was used to author content, with assistance strictly limited to editing, formatting, and error correction.

⁸<https://www.claude.ai>

Erklärung

Hiermit erkläre ich, dass ich die vorliegende Arbeit selbstständig verfasst und dabei keine anderen als die angegebenen Quellen und Hilfsmittel benutzt habe. Sämtliche wörtlichen oder sinngemäßen Übernahmen und Zitate aus anderen Werken sind kenntlich gemacht und nachgewiesen. Ich versichere, dass ich keine Hilfsmittel verwendet habe, deren Nutzung die Prüferin oder der Prüfer explizit ausgeschlossen hat.

Im Anhang „Nutzung von KI-Tools“ habe ich dokumentiert, inwiefern KI-Tools zur Erstellung wesentlicher Teile dieser Arbeit verwendet wurden. Mit Abgabe der Arbeit übernehme ich die volle Verantwortung für alle Inhalte. Die *Grundsätze zur KI-Nutzung für Studierende an der Universität Stuttgart* habe ich zur Kenntnis genommen und befolgt.

Weder diese Arbeit noch wesentliche Teile daraus waren bisher Gegenstand eines anderen Prüfungsverfahrens. Ich habe diese Arbeit bisher weder teilweise noch vollständig veröffentlicht. Das elektronische Exemplar stimmt mit allen eingereichten Exemplaren überein.

Datum, Unterschrift der Studentin / des Studenten

Declaration

I hereby declare that the work presented in this thesis is entirely my own. I did not use any other sources and aids than the listed ones. I have marked all direct or indirect transfers from other sources contained therein as referenced quotations. I assure that I have not used any aids that were explicitly ruled out by the examiner.

In Appendix "Use of AI Tools" I have documented to what extent AI tools were used for generating any significant parts of this work. By submitting this work, I assume full responsibility for all content. I have taken notice of and I comply with the *Principles for the Use of Artificial Intelligence by Students at the University of Stuttgart*.

Neither this work nor significant parts of it were part of another examination procedure. I have not published this work in whole or in part before. The electronic copy is consistent with all submitted hard copies.

Date and signature of the student